Safe to Drop: an Exact Criterion for Quantum Temporaries

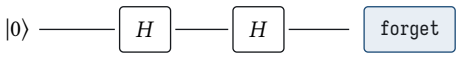
ANONYMOUS AUTHOR(S)

CCS Concepts: • **Software and its engineering → Formal language definitions**; • **Theory of computation** → *Programming logic*.

Additional Key Words and Phrases: quantum programming languages, uncomputation, ancilla, deallocation, dynamic lifting, type systems, contextual equivalence

1 Introduction

Consider a simple two-gate quantum circuit. It allocates a qubit, applies a Hadamard, applies a second Hadamard, and asks the compiler to forget it:



$H \cdot H = I$: the register is $|0\rangle$ again

Reusing a temporary avoids a fresh allocation and can reduce circuit width. Uncomputation restores its allocation state, usually $|0\rangle$, by reversing its producer. We ask when that whole reversal is unnecessary. Our checker finds the fixed pure state left on a safe temporary and the compiler then constructs one local gate that restores it. Reuse preserves the program's answers (proposition 4.13).

In Silq [6] the circuit above is the four line program `t0 := 0:B; t1 := H(t0); t := H(t1); forget(t)`. The register is $|0\rangle$, two Hadamards composing to the identity, so discarding it is safe, and Silq agrees if the programmer says so with `forget(t=0:B)`; the automatic `forget(t)` is refused with *cannot synthesize forget expression.*

Silq and Unqomp rely on a `qfree` producer, one that creates no superposition; H fails this condition [6, §4.1]. Unqomp uses $H; H$ to illustrate its incompleteness [32]. Q# requires a released qubit to be back in the state it was allocated in [31, 39], and Quipper's `with_ancilla` closes on the programmer's promise of the same [15]. Program (a) below instead leaves its temporary away from $|0\rangle$. ReQWIRE calls its syntactic checks sufficient but not necessary, and Qutes calls its detection incomplete [11, 35].

We express safety through program equivalence. A temporary is safe to drop when the computation so far is indistinguishable from one that prepares it once and never uses it again. With measurement, the two must also agree on each outcome record and its probability, called its weight. That witness may measure other registers. This criterion differs from circuit-level checks of restoration, extracted functionality, and unchanged loans [28, 29, 45]; section 9 gives the precise comparisons.

Three programs

Our language (section 3) types both safe and unsafe drops: `drop` consumes register names without a safety premise. In the examples, t is allocated by the program and v is its input and returned register. Both are qubits; later we allow arbitrary finite dimensions and tuple drops.

Example 1.1 (the running examples).

(a) *superpose, then drop*

```
prog a (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t;

  drop t;
  return v }
```

(b) *superpose, then read*

```
prog b (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t;
  CX t v;

  drop t;
  return v }
```

(c) *copy, measure, one joint gate*

```
prog c (v : [2]) : [2] {
  new t : [2] @ |0>;
  H v;  CX v t;
  p = lift[{0}] t;
  V t v;
  drop t;
  return v }
```

Program (a) superposes the temporary and drops it. Its final state is $|+\rangle$ whatever the input: constant, uncorrelated with v, and not $|0\rangle$.

Program (b) reads the temporary into v before dropping it. At input $|0\rangle_v$, the two registers form a Bell pair. The temporary is then entangled with v, so no gate on the temporary alone can make it reusable.

A gate in this language is any unitary matrix (definition 3.1). Program (c) superposes v, copies it into t, measures whether t is in $|0\rangle$, and applies the same joint gate V on either outcome. In the basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, with t first, it is

$$V = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1/\sqrt{2} & 0 & 1/\sqrt{2} \\ 0 & -1/\sqrt{2} & 0 & 1/\sqrt{2} \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Both outcomes have probability one half at every basis input. After V, one outcome leaves a safe drop and the other an unsafe one. The program text determines both verdicts; the outcome determines which applies (section 5.1).

Why discard is not free here

The quantum λ-calculus also permits affine use of quantum data [37]. In a channel semantics, discarding a register means taking a partial trace, which is defined for every state of that register [36]. This operation describes what remains observable, but it does not certify that the discarded physical register is ready for another allocation.

Our machine keeps dropped registers as hidden state factors; `drop` applies no physical operation. The compiler can reclaim a safe block, the tuple of registers one drop names. Our exact matrix test computes the site map on a basis of inputs; its size can be exponential in the number of qubits. Enumerating measurement records can add another exponential cost (section 7).

Section 8 evaluates the implementation on nineteen typed programs with twenty-one drop sites.

2 The idea, on three programs

2.1 The temporary is the scrutinee of an invisible switch

Consider a measurement-free program with a temporary of dimension d. We can separate its action into d components, indexed by the temporary's basis state, much like the branches of a `switch`. The ith component is an $m \times n$ matrix K_i: it maps the input space, of dimension n, to the rest of the machine, of dimension m. The rest includes both live and previously hidden registers. The rows of the program map K labelled by the temporary's level i give K_i:

$$Kv \;=\; \sum_{i=0}^{d-1} |i\rangle \otimes K_i v, \qquad K_i = (\langle i| \otimes I)\, K.$$

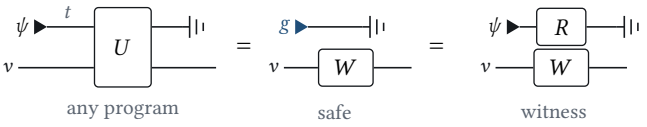


Fig. 1. The measurement-free criterion. A triangle prepares a state; a ground symbol marks `drop`. **Left:** an arbitrary unitary program. **Middle:** the safe shape (definition 4.1): the temporary comes out in one state g whatever the input was, and nothing crosses between the registers. **Right:** the witness shape (definition 4.7): prepared by one gate R and dropped, never having met v. Left equals middle exactly when the drop is safe, and by theorem 4.10 exactly when it equals right. With measurement, the witness may also need an indicator qubit (theorem 5.2).

A density matrix ρ is positive semidefinite with trace one. It is pure when $\rho = |\psi\rangle\langle\psi|$ for a unit vector ψ. Tracing out a register gives the reduced state of those retained. The K_i are the *Kraus operators* [25] of the channel that dropping the temporary induces on the rest, $\rho \mapsto \sum_i K_i \rho K_i^\dagger$. Quantum mechanics adds one thing to the switch: every branch runs, and the results are kept side by side, each labelled by its branch.

The drop hides the label. It is safe exactly when these components are all proportional to one another: $K_i = g_i W$ for one map W and one unit vector g. Equivalently, the induced channel has Kraus rank at most one. At a safe measurement-free site, K preserves norm and is nonzero, so the induced channel has Kraus rank exactly one. A zero measurement-record map is also safe.

2.2 Program (a) is safe, and it does not restore anything

A *site* is an occurrence of `drop`; a one-drop program is called safe when its site is. Here are the maps of programs (a) and (b), with rows grouped by the temporary's level:

Program (a) sends v to $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes v$:

$$K_{(a)} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{matrix} \} K_0 = \frac{1}{\sqrt{2}} I \\ \} K_1 = \frac{1}{\sqrt{2}} I \end{matrix}$$

Both components are $I/\sqrt{2}$, so the drop is safe. Its fixed temporary state $|+\rangle$ is its *certificate*.

Program (b) sends v to $\frac{1}{\sqrt{2}}(|0\rangle \otimes v + |1\rangle \otimes Xv)$:

$$K_{(b)} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{matrix} \} K_0 = \frac{1}{\sqrt{2}} I \\ \} K_1 = \frac{1}{\sqrt{2}} X \end{matrix}$$

The two branches act differently, and the drop is unsafe.

After the drop, program (a) leaves the visible input state ρ unchanged; program (b) leaves $\frac{1}{2}(\rho + X\rho X)$. The safe program has a witness that prepares the temporary once and never couples it to the input: in fig. 1, $R = H$ and $W = I$. The formal equivalence is theorem 4.10.

3 The language

The language is first order, with no host language, procedures or recursion. A measurement-free program is a circuit; measurement later adds classical bits, branching, and a tree of runs.

3.1 Syntax

In fig. 2, $[d]$ is a register type with $d \geq 2$; the unit type 1 holds no register. The type interpretation is $[\![1]\!] = \mathbb{C}$, $[\![[d]]\!] = \mathbb{C}^d$ and $[\![A \otimes B]\!] = [\![A]\!] \otimes [\![B]\!]$. A context denotes the tensor product of its register spaces. These spaces are finite-dimensional complex Hilbert spaces throughout. The constants come from two parameters, a *scalar domain* $\mathbb{K} \subseteq \mathbb{C}$ closed under conjugation, and a *signature* $\mathcal{G}$, a set of unitaries with entries in $\mathbb{K}$. A program's gates U are drawn from $\mathcal{G}$, and its allocation states ψ are basis states $|i\rangle$ or named unit vectors over $\mathbb{K}$, which `new` prepares the

types	$A ::= 1 \mid [d] \mid A \otimes B$	no register; a d-level register; two side by side
values	$v ::= x \mid () \mid (v_1, v_2)$	a register name; nothing; a pair
terms	$M ::=$ `return` v	hand back the registers v names
	$\mid$ $U\,v$; M	a gate on the registers v names, in that order
	$\mid$ `new` $t : [d]$ `@` ψ; M	allocate t in the unit state ψ
	$\mid$ `drop` v; M	forget the registers v names
programs	`prog` $f\,(x_1 : [d_1], \ldots, x_n : [d_n]) : A\,\{M\}$	an input may be marked `loan` $a : [d]$, lent by the caller
added with measurement (section 5)		
	$M ::= \cdots \mid p = \mathtt{lift}_S\, x$; M	measure whether $x \in S \subseteq [d]$; x stays live under its own name
	$\mid$ `if` $e\,\{M_1\}$ `else` $\{M_2\}$	classical control on a lifted bit, in tail position
guards	$e ::= b \mid p \mid \neg e \mid e_1 \wedge e_2 \mid e_1 \vee e_2$	Boolean expressions; $b \in \{0, 1\}$

Fig. 2. The language. A term is a sequence of statements ending in `return`, the spelling a program file uses.

register in. This section and the two after it take $\mathbb{K} = \mathbb{C}$ and $\mathcal{G}$ every unitary. Later in section 6 we restrict the scalars to specific rings and the gates accordingly. The checker computes in the tower $\mathbb{Q}(\zeta_{2^k})$, exactly, with the gate table of section 3.6. A *loan*, written `loan a : [d]` in the header, is an input lent by the caller in an unknown state. It is typed and run like any other input. A separate well-formedness condition requires it to be dropped exactly once on every branch. That condition only checks where the name is consumed. The safety criterion checks whether a gate on the dropped block can return the loan in its original state. At an unsafe drop the compiler emits no such gate; the loan remains a hidden factor in the machine's state (sections 3.6 and 4.1).

A gate and a `drop` bind nothing, and a `lift` binds only its parameter, so `new` is the only register binder of the language and every register keeps the name it was allocated under. The freshness discipline is one line: a rule that needs a name fresh takes one, and terms are identified up to renaming of the names `new` binds.

Deutsch–Jozsa on one input qubit, with the oracle $f(x) = x$, is the whole language at work.

```
prog dj (x : [2]) : [2] {
  new t : [2] @ |0>;
  X t;  H t;   - t := |->
  H x;  CX x t;  H x;   - the oracle
  drop t;
  return x
}
```

A *temporary* is a register the program itself allocates, as against an input it is handed. Here t is the only temporary; an X and a Hadamard put it in $|-\rangle$, the oracle kicks a phase back onto `x`, and the drop forgets it in $|-\rangle$ — not in the state it was allocated in. Section 4 decides whether that drop is safe.

Two surface forms are expanded before typing, and one operator expands them. We write $M_1 \mathbin{+\!\!+}_{\bar{y}} M_2$ for "run M_1, bind the value it returns to the tuple of names $\bar{y}$, continue with M_2", by recursion on M_1: one clause for `return`, one for `if`, and one shared by the four statement forms of fig. 2, written *stmt*:

$$(\mathtt{return}\ v) \mathbin{+\!\!+}_{\bar{y}} M \;=\; M[v/\bar{y}], \qquad (\mathit{stmt};\, M') \mathbin{+\!\!+}_{\bar{y}} M \;=\; \mathit{stmt};\, (M' \mathbin{+\!\!+}_{\bar{y}} M),$$
$$(\mathtt{if}\ e\{M_1\}\ \mathtt{else}\ \{M_2\}) \mathbin{+\!\!+}_{\bar{y}} M \;=\; \mathtt{if}\ e\{M_1 \mathbin{+\!\!+}_{\bar{y}} M\}\ \mathtt{else}\ \{M_2 \mathbin{+\!\!+}_{\bar{y}} M\}.$$

The binder $\bar{y}$ is a *tuple of names* which is a value all of whose leaves are names because that is the only pattern that ever occurs: a binder of register type is a name and one of unit type is $()$. Substituting a tuple of names for a tuple of names is just renaming, so the language has no substitution beyond renaming. The surface form $\bar{y}$ `= { }` M_1 `; }` M_2 uses braces to scope a subprogram. Its result is bound to the tuple of names $\bar{y}$; the whole form expands to $M_1 \mathbin{+\!\!+}_{\bar{y}} M_2$. This use of braces is distinct from a dropped *block*, the registers named by one `drop`. A non-tail `if` expands to $(\mathtt{if}\ e\{M_1\}\ \mathtt{else}\ \{M_2\}) \mathbin{+\!\!+}_{\bar{y}} M$. The last clause copies the continuation into both branches; the

artifact compiles the expanded term, so it can emit the continuation more than once. Expansion can grow exponentially under nested conditionals, without changing any run, site or verdict.

3.2 Statics

A *register context* Ω is a finite set of typed register names $x : [d]$ with pairwise distinct names, and the comma Ω_1, Ω_2 is defined only when the two have disjoint domains. That one convention is the whole discipline: no register is named twice, and none is discarded without a `drop`. Two judgements share the one context: $\Omega \vdash v : A$ for values and $\Omega \vdash M : A$ for terms. A statement is not a term of the language, and nothing types one alone.

Definition 3.1 (the language). The two judgements are defined inductively by these rules.

$$\frac{}{x : [d] \vdash x : [d]}\ \textsc{Var} \qquad \frac{}{\cdot \vdash () : 1}\ \textsc{Unit} \qquad \frac{\Omega_1 \vdash v_1 : A \quad \Omega_2 \vdash v_2 : B}{\Omega_1, \Omega_2 \vdash (v_1, v_2) : A \otimes B}\ \textsc{Pair}$$

$$\frac{\Omega \vdash v : A}{\Omega \vdash \texttt{return}\ v : A}\ \textsc{Return} \qquad \frac{U \in \mathcal{G} \text{ on } [\![A]\!] \quad \Omega_1 \vdash v : A \quad \Omega_1, \Omega_2 \vdash M : B}{\Omega_1, \Omega_2 \vdash U\, v;\, M : B}\ \textsc{Gate}$$

$$\frac{\psi \in \mathbb{K}^d,\ \|\psi\| = 1 \quad \Omega, t : [d] \vdash M : B}{\Omega \vdash \texttt{new}\ t : [d] \mathbin{@} \psi;\, M : B}\ \textsc{New} \qquad \frac{\Omega_1 \vdash v : A \quad \Omega_2 \vdash M : B}{\Omega_1, \Omega_2 \vdash \texttt{drop}\ v;\, M : B}\ \textsc{Drop}$$

A *program* `prog` $f\,(x_1 : [d_1], \ldots, x_n : [d_n]) : A\,\{M\}$ is the judgement $x_1 : [d_1], \ldots, x_n : [d_n] \vdash M : A$ in which every input marked `loan` is dropped exactly once on every branch, an `if` dropping it in both arms; the marker asks nothing else, a loan being typed as any other input.

Gate keeps the registers it acts on: they are in the context before the statement and in the context after it, under the same names, which is why the rule has no binder. New extends the context by one fresh name and the restriction $d \geq 2$ excludes a register of dimension one, whose every drop is safe with its one basis vector as certificate. Drop consumes the context that named its value and carries no premise about safety (section 1). It takes a tuple as readily as a name, and the two differ (section 4.1).

The types reject use after a drop, duplicated operands, an unreturned and undropped temporary, and a gate applied at the wrong type or arity. With branching, they also enforce matching return types and parameter scope. They permit unsafe drops. A polynomial type checker cannot decide safety exactly unless P = NP (theorem 7.1). For example, this program type-checks:

```
prog f (x : [2]) : [2] {
  new t : [2] @ |0>; CX x t;
  drop x; return t
}
```

It copies the input's basis value into a temporary and drops the input. At a superposed input the two registers are entangled, so the drop is unsafe (section 4).

3.3 Dynamics

A *configuration* $\langle \Delta, \sigma \parallel M \rangle$ carries the registers Δ allocated so far in allocation order, a vector $\sigma \in [\![\Delta]\!]$, and a term M. The *hidden* registers are the allocated ones the term no longer names and along a run that begins with every allocated register in use — the run of theorem 3.3(4) — they are exactly the ones the run has dropped, since a register leaves the term's context at the `drop` that consumes it and nowhere else. At a `drop` v the registers v names are its *block*; every other register

allocated so far, live or hidden, is dually its *rest*. A block holds one register or several: every verdict below is a statement about a block, every certificate a state of a block, and every reclaiming gate acts on a block and nothing else. Three rules fire, each at the head of M (fig. 3).

$$\frac{}{\langle \Delta, \sigma \parallel U\, v;\ M\rangle \ \longmapsto\ \langle \Delta, U_v\, \sigma \parallel M\rangle}\ \textsc{Step-Gate}$$

$$\frac{w \notin \Delta \quad w \text{ is not bound in } M}{\langle \Delta, \sigma \parallel \texttt{new}\ t\ :\ [d]\ @\ \psi;\ M\rangle \ \longmapsto\ \langle \Delta{\cdot}w, \sigma \otimes \psi_w \parallel M[w/t]\rangle}\ \textsc{Step-New}$$

$$\frac{}{\langle \Delta, \sigma \parallel \texttt{drop}\ v;\ M\rangle \ \longmapsto\ \langle \Delta, \sigma \parallel M\rangle}\ \textsc{Step-Drop}$$

Fig. 3. The measurement-free machine. U_v is U acting on the factors of $[\![\Delta]\!]$ that v names, in the order v lists them; ψ_w is ψ on the fresh register w. Step-New's second side condition is needed because without it the renaming $M[w/t]$ captures a binder of M and the successor clones a register.

A configuration is *final* when M is `return` v. Step-Drop applies nothing to σ and allocates nothing: it deletes a statement. The paper's whole difficulty is in that one rule: the type says the register is gone, the state says it is not, and section 4 says when the two agree.

3.4 What a program denotes

A measurement-free program has one run up to fresh register names, so its meaning is one map rather than a family. Write $\mathcal{H}_{\mathsf{hid}}$, or $\mathcal{H}^{M}_{\mathsf{hid}}$ when the program must be named, for the tensor product of the spaces of the registers M drops. Then a typed program denotes

$$[\![M]\!]\ :\ [\![\Omega]\!] \longrightarrow \mathcal{H}_{\mathsf{hid}} \otimes [\![A]\!],$$

the map whose clauses are the machine's rules read as linear maps: a gate contributes its unitary, `new` adjoins ψ, `drop` contributes the identity — the register moves from the output factor to $\mathcal{H}_{\mathsf{hid}}$, and nothing is applied to it — and a statement sequence composes. Every clause preserves norm, so $[\![M]\!]$ is an *isometry*. We retain its hidden output explicitly. In the construction $L[\mathsf{Isometry}]$ of quantum information effects [17], such isometries represent channels after quotienting away the choice of hidden output. We use the representatives, not just the resulting channels, because safety concerns the registers being hidden.

3.5 Type safety

A configuration is well typed, $\vdash \langle \Delta, \sigma \parallel M\rangle\ :\ A$, when $\sigma \in [\![\Delta]\!]$, the names of Δ are distinct, and $\Omega_M \vdash M\ :\ A$ where Ω_M is the part of Δ that M still names. One fact about names is needed.

Remark 3.2 (renaming). Let $\Omega \vdash M\ :\ A$ and let w be a name absent from Ω and *not bound in* M. Then $\Omega[x \mapsto w] \vdash M[w/x]\ :\ A$, by induction on the derivation: every rule's context is renamed along, and the disjointness premises of Pair, Gate and Drop survive an injective renaming — injective on the free names because w is not in Ω. Only New binds and there the renaming stops if the binder is x, and otherwise descends, because w is not bound in M.

Theorem 3.3 (type safety). *Let $\vdash \langle \Delta, \sigma \parallel M\rangle\ :\ A$ be a well-typed configuration of the measurement-free machine.*

(1) Progress. *Either M is* `return` v*, and the configuration is final, or exactly one rule applies to it, and the successor is determined once the name* Step-New *allocates is fixed.*

(2) Preservation. *The step yields a well-typed configuration at the same type A.*

(3) Termination. *Every run from the configuration reaches a final configuration in exactly $|M|$ steps, where $|M|$ counts its statements and does not count* `return`*.*

(4) Adequacy. *The run from $\langle \Omega, x \parallel M \rangle$, the input registers allocated and the names it allocates fresh for M, ends in the state $[\![M]\!]\, x$.*

Canonical forms is immediate: inverting the three value rules, a value of type $[d]$ is a register name, one of type 1 is $()$, and one of type $A \otimes B$ is a pair.

The sequencing operator discharges the surface forms and preserves typing and meaning.

Lemma 3.4 (desugaring). *Suppose $\Omega_1 \vdash M_1 : A$, $\Omega_{\bar{y}} \vdash \bar{y} : A$, and $\Omega_2, \Omega_{\bar{y}} \vdash M_2 : B$. The three contexts are pairwise disjoint. Every name bound in M_1 (register or parameter) is fresh for M_2, $\bar{y}$ and Ω_2. Every name bound in M_2 is fresh for Ω_1 and $\bar{y}$.*

(1) $\Omega_1, \Omega_2 \vdash M_1 +\!\!+_{\bar{y}} M_2 : B$.

(2) *Every run of M_1 to* `return` v*, followed by the run of $M_2[v/\bar{y}]$ from the state it reaches, is a run of $M_1 +\!\!+_{\bar{y}} M_2$: same record and state. In the measurement-free case,*

$$[\![M_1 +\!\!+_{\bar{y}} M_2]\!] = (\mathrm{id}_{\mathcal{H}_{\mathrm{hid}}^{M_1}} \otimes [\![M_2]\!]) \circ ([\![M_1]\!] \otimes \mathrm{id}_{[\![\Omega_2]\!]}).$$

(3) $|M_1 +\!\!+_{\bar{y}} M_2| = |M_1| + |M_2|$. *Here $|M|$ counts statements along the longest branch of M:* `return` *counts zero, each other statement counts one, and an* `if` *counts one plus the larger count of its two arms.*

The disjointness of $\Omega_{\bar{y}}$ from Ω_1 is not decoration: without it the componentwise renaming can clone a register. The converse of (1) is false — the continuation may name a register of Ω_1 directly.

The proofs of section 3 are in section A.

3.6 Compilation to circuits

The *parser* reads program text and builds the term of fig. 2. The *emitter* translates that term into OpenQASM 3 instructions. It supports fewer gates than the parser: accepting a program's syntax does not promise that this compiler can produce its circuit.

The compiler represents a register of type $[2^k]$ by k qubits. It refuses other dimensions; it neither truncates a register nor embeds it into a larger one. Its supported gates are H, X, Z, S, T, cnot, Toffoli, swap and Grover's diffusion. The last is written `Diff` in source and `diff` in OpenQASM; on its two qubits the source matrix is $2|{+}{+}\rangle\langle{+}{+}| - I$. The emitter also handles three literals: `perm` for a permutation of basis states, `diag` for a diagonal unitary, and `mat` for a one-qubit unitary. It writes permutations as transpositions, diagonal gates as controlled phases, and one-qubit matrices as $U(\theta, \varphi, \lambda)$ gates. A gate outside these cases is refused.

Braced subprograms are expanded before emission using lemma 3.4. For example, the two fragments below produce the same term; the returned name t replaces a in the continuation:

```
a = { new t : [2] @ |0>; return t };     ⟼     new t : [2] @ |0>;
H a; drop a; return ()                   ⟼     H t; drop t; return ()
```

A basis-state allocation `new t : [`2^k`] @ |`i`⟩` reserves k fresh physical qubits and applies X to the set bits of i. A loan has no allocation instruction: its physical qubits are supplied by the caller, in an unknown state. They can be entangled with registers the caller keeps.

What is inserted at a drop. At an unsafe drop the compiler inserts no gate and leaves the register physically allocated. It does not reclaim that register. At a safe drop it inserts a *reclaiming gate*: a gate on the dropped block that restores its temporaries to their allocation states and its loans to their original states. The empty drop emits nothing. The following constructions say which safe sites this emitter can handle; section 4 proves why they are correct.

For a block of temporaries, let ψ be its allocation state and $[g]$ the fixed pure state left at the drop, its *certificate ray*. Choose its unit representative $\tilde{g}$ so that $\langle\psi, \tilde{g}\rangle$ is nonnegative real. If this inner product is zero, choose the first nonzero coordinate of $\tilde{g}$ positive real. This fixes the representative before constructing the gate. If $\tilde{g} = \psi$, no gate is needed. Otherwise put $u = (\psi - \tilde{g})/\|\psi - \tilde{g}\|$. The reclaiming gate is $R^s = I - 2uu^\dagger$, a reflection that exchanges $\tilde{g}$ and ψ.

Figure 4(a) implements this reflection by carrying its axis u to a basis state, changing that basis state's sign, and undoing the first step. The state-preparation circuit $V^\dagger$ is built from controlled rotations for the magnitudes and controlled phases for the relative phases. The construction uses $O(2^k)$ controlled gates on k qubits. The reflection circuit uses OpenQASM's built-in U gate and gates from `stdgates.inc`, possibly with `ctrl` or `negctrl`. The device's transpiler must still decompose those controls. Source programs may also call `diff`; the emitted file declares that custom gate using H, X and controlled Z.

Panel (b) gives a two-qubit example. Allocate t_1, t_2 in $|00\rangle$, then run `H t1; CX t1 t2; drop (t1,t2)`. The Hadamard prepares a superposition on t_1; the controlled negation correlates t_2 with it, giving the Bell state $(|00\rangle + |11\rangle)/\sqrt{2}$. The pair is a fixed block, so its joint drop is safe even though neither qubit is separately fixed. The displayed five-gate circuit is the emitter's reflection for this certificate. Reversing the two source gates would also restore this particular state; the figure illustrates the general reflection construction.

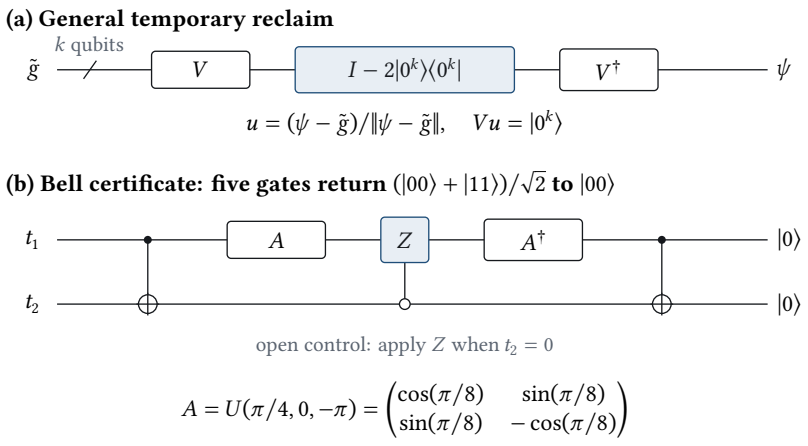


Fig. 4. Reclaiming acts only on the dropped block; time runs left to right. (a) Conjugating one basis-state phase flip gives $R^s = V^\dagger(I - 2|0^k\rangle\langle 0^k|)V$. (b) For the Bell certificate and allocation $|00\rangle$, the axis is $\sin(\pi/8)|00\rangle - \cos(\pi/8)|11\rangle$. The first CNOT and A carry it to $|10\rangle$, so the middle gate flips exactly that basis state. The final two gates undo the first two.

There are shorter cases. A nonidentity one-qubit reclaim uses one U gate. For allocation $\psi = |0\rangle$ and the representative $\tilde{g} = (a, b)$ fixed above, it is $U(2\arccos a, \arg b, \pi - \arg b)$, exactly R^s. Here $b \neq 0$; the case $b = 0$ needs no gate. At $a = 0$ the tie-break makes $b = 1$, so the line is X. For allocation $|1\rangle$ and $0 < \langle\psi, \tilde{g}\rangle < 1$, the emitted line is $-R^s$; this global sign changes no state. If ψ

and $\tilde{g}$ are distinct basis states on a wider block, R^s exchanges them and fixes the others. An axis supported on two basis states uses CNOTs to align them on one qubit and then one rotation, a controlled Z, and the inverse steps, as in fig. 4(b).

Returning loans. A safe block containing only loans has site map $B \otimes W$: B is the unitary action on the loans and W acts on the other inputs. The *block factor* B is undone by $B^\dagger$. A loan needs this gate whenever the program changed it; being dropped removes its name, not the change to its state. This emitter handles $B = I$ with no gate, a one-qubit B with one U gate, and a basis permutation by the inverse permutation.

If the block also contains temporaries, B maps the loan-input space into a larger block space, so its matrix is rectangular. Figure 5 shows a two-dimensional input and a four-dimensional block. The emitter handles the product form $B = B' \otimes g$ after grouping loans and temporaries, with B' unitary and g unit: it applies $B'^\dagger$ to the loans and the reflection to the temporaries. These operations use the original tuple order, even when the two kinds of register are interleaved. The supported B' are the same three cases just listed. An arbitrary isometric B can also be undone by a unitary on the whole block (proposition 4.12); this emitter does not synthesise that unitary in general. Failure to emit it does not mean that the site is unsafe.

For a concrete mixed block, the program in fig. 5 borrows a qubit a and allocates a temporary t in $|0\rangle$. The instruction `H t` prepares $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$; `X a` flips the loan. Thus at `drop (a,t)`, an input $\ell = \alpha|0\rangle + \beta|1\rangle$ has become $(\beta|0\rangle + \alpha|1\rangle) \otimes |+\rangle$. The block contains both registers, in the order (a, t). Restoring it takes two gates: X undoes the loan's flip and H sends $|+\rangle$ back to the temporary's allocation state $|0\rangle$.

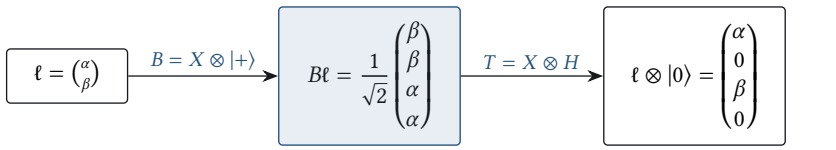
Source program

```
prog mixed (loan a : [2]) : 1 {
  new t : [2] @ |0>;
  H t; X a;
  drop (a,t); return ()
}
```

Inserted just before the drop

```
X a; H t;
```

These gates return a to ℓ and t to $|0\rangle$. The source's `drop` itself applies no gate.



$$B = \frac{1}{\sqrt{2}}\begin{pmatrix}0 & 1\\ 0 & 1\\ 1 & 0\\ 1 & 0\end{pmatrix}, \qquad B^\dagger B = I_2, \qquad TB = I_2 \otimes |0\rangle.$$

Block basis order: $|00\rangle, |01\rangle, |10\rangle, |11\rangle$; loan first.

Fig. 5. The code explains both arrows. Allocation followed by `H t; X a` gives the block map B; the inserted `X a; H t` gives T. Read the vectors in the order (a, t). The rectangular matrix records two possible loan inputs in a four-dimensional block. Its columns are orthonormal, and $TB\ell$ is the original loan beside a fresh temporary.

Domain and correctness. The *compilation fragment* has power-of-two register dimensions, basis-state allocations and the supported gates listed above. Non-basis allocations receive only a comment requesting external preparation, so they are outside this fragment. Conditionals may vary gates,

but allocation, lift and drop sites must be independent of the branch. Every branch returns the original non-loan inputs in their declared order. Each allocation uses fresh physical qubits; this is the emitter's circuit, not the pooled machine of proposition 4.13. Lifts test computational-basis subsets. A one-qubit test is measured directly; wider tests use the indicator construction of remark 5.4. Circuit maps are compared before terminal output measurements. The implemented loan path is measurement-free. A safe site is *emittable* if its block has only temporaries, or if its loan factor has one of the supported shapes described above.

OpenQASM's `gphase` instruction would multiply the entire state vector by a scalar of modulus one. This emitter does not write that instruction. Converting a matrix to a U gate can therefore omit such a *global phase*; the `diff` expansion can do so too. Changing the representative in a mixed-block split can contribute another phase. These scalars are fixed by the program and its sites, independently of the input. The compilation theorem keeps them explicit.

Theorem 3.5 (compilation). *Let M be a measurement-free program in the compilation fragment. Assume each safe site $s_1, \dots, s_r$ is emittable. Write E_M for the emitted circuit's map and R^{s_j} for the ideal reclaiming gate at s_j.*

(1) Full state. *There is one scalar z, $|z| = 1$, such that*

$$E_M = z\,(R^{s_1} \otimes \cdots \otimes R^{s_r} \otimes I) \circ [\![M]\!].$$

The output factors are grouped by dropped block, in site order, followed by the remaining factors. The scalar z is independent of every input, including the states of the loans; it is the product of the phases omitted during emission.

(2) Returned output. *The emitted circuit and M have the same reduced state on the returned registers at every input. If no reclaiming gate is inserted, their full maps differ only by z.*

(3) Reusable registers. *Each reclaimed temporary is in its allocation state. Each reclaimed loan is in its original state, with its correlations with the caller preserved. These are equalities of states, so the global phase in (1) does not affect them.*

The theorem concerns ideal gates with exact angles. The artifact computes angles numerically and uses numerical thresholds to select emission cases; its simulations test those choices, rather than proving exact arithmetic. The site's safety verdict is computed separately using exact scalars (section 8). For a one-qubit matrix V, the ideal emitted U gate is $e^{-i\delta}V$, where $\delta = \arg V_{00}$ if $V_{00} \neq 0$ and $\delta = \arg V_{10}$ otherwise. It equals V exactly when that selected entry is positive real. Individual instruction phases may cancel, so exact instructions suffice for, but are not necessary for, equality of the full circuit maps. Coherent control of the whole compiled circuit would make its global phase observable; that operation is outside this language.

3.7 Four algorithms, as programs

Deutsch–Jozsa appears in section 3.1. The next two programs use a known temporary. Grover computes the predicate $x_1 x_2$ into t, uses it for a phase, then uncomputes it. Thus t returns to $|0\rangle$. Phase estimation instead puts ψ in the T gate's eigenstate $|1\rangle$. The three controlled powers change only the counting registers, so ψ stays on $|1\rangle$ and is safe to drop despite not returning to $|0\rangle$. The remaining gates implement the inverse Fourier transform; `z8` denotes $e^{i\pi/4}$.

```
prog grover (x1 : [2], x2 : [2])
  : [2] * [2] {
  H x1; H x2;
  new t : [2] @ |0>;
  CCX x1 x2 t;
  Z t;
  CCX x1 x2 t;
  drop t;
  Diff x1 x2;
  return (x1, x2)
}
```

```
prog qpe (c1 : [2], c2 : [2], c3 : [2])
  : [2] * [2] * [2] {
  new psi : [2] @ |0>; X psi;
  H c1; H c2; H c3;
  diag c1 psi (1, 1, 1, -1);
  diag c2 psi (1, 1, 1, i);
  diag c3 psi (1, 1, 1, z8);
  Swap c1 c3; H c3;
  diag c2 c3 (1, 1, 1, -i); H c2;
  diag c1 c3 (1, 1, 1, -z8^3);
  diag c1 c2 (1, 1, 1, -i); H c1;
  drop psi; return (c1, c2, c3)
}
```

The fourth program uses a loan a whose state is unknown.

Each Toffoli flips its third register when its first two are 1. For basis input (x_1, x_2, x_3, y, a), the table tracks the two registers that change; q_1, q_2, q_3 stay fixed.

```
prog mcx3 (q1 : [2], q2 : [2],
  q3 : [2], q : [2], loan a : [2])
  : [2] * [2] * [2] * [2] {
  CCX q3 a q;
  CCX q1 q2 a;
  CCX q3 a q;
  CCX q1 q2 a;
  drop a;
  return (q1, q2, q3, q)
}
```

	q	loan a
input	y	a
gate 1	$y \oplus x_3 a$	a
gate 2	$y \oplus x_3 a$	$a \oplus x_1 x_2$
gate 3	$y \oplus x_1 x_2 x_3$	$a \oplus x_1 x_2$
gate 4	$y \oplus x_1 x_2 x_3$	a

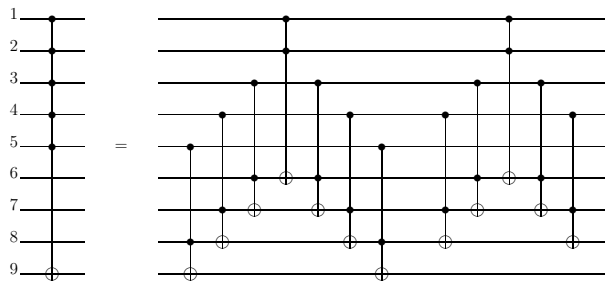


Fig. 6. Barenco et al.'s original circuit for five controls, reproduced from [3, Lemma 7.2]. Registers 6–8 are borrowed and returned unchanged; register 9 is the target. The program above is its three-control case: one loan and four Toffolis.

The two occurrences of $x_3 a$ cancel. The data therefore undergo C^3X and the loan is unchanged. Linearity gives the same identity for superpositions and for a loan entangled with the caller. This is the three-control instance of Barenco et al.'s construction (fig. 6) [3, Lemma 7.2]. The compiler emits these four Toffolis on the five supplied inputs, including the caller's loan; no reclaiming gate is needed.

4 The site and the criterion

Longer proofs are in section B.

4.1 The drop site

In Deutsch–Jozsa, the state at the drop is $|-\rangle_t \otimes X|x\rangle$: the block being dropped is on the left and the rest is on the right. A site records this split immediately before the drop. Figure 7 gives a separate example with several allocations, a tuple drop, and a previously hidden register.

```
prog order (x : [2]) : [2] {
  new t : [2] @ |0>;
  new h : [2] @ |0>;
  new a : [2] @ |0>;
  drop h;
  drop (a,t);
  return x
}
```

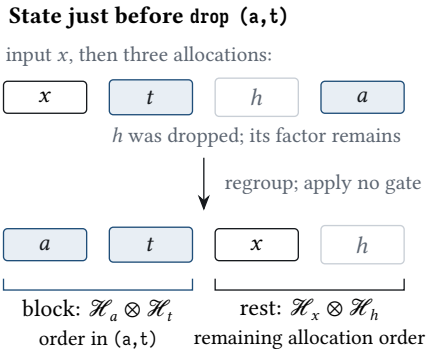


Fig. 7. At `drop (a,t)`, the block is (a,t) and the rest is (x,h). Regrouping changes the written order, not the state; hidden h is retained.

Definition 4.1 (the site, and safety at it). Let $\Omega \vdash M : A$ be a program.

(1) A *site* s of M is an occurrence of `drop` v in it. Its *block* is the ordered list of registers v names — none for $v = ()$, a site with $\mathcal{H}_v = \mathbb{C}$, safe with the unit scalar as certificate; $\mathcal{H}_v$ is the tensor product of their spaces in that order. The space $\mathcal{H}_{\text{rest}}$ is the tensor product of all remaining register spaces: inputs in context order, followed by allocations, including previously dropped registers (section 3.3).

(2) The *site map* $\sigma_s : [\![\Omega]\!] \to \mathcal{H}_v \otimes \mathcal{H}_{\text{rest}}$ sends an input vector $\xi \in [\![\Omega]\!]$ to the joint state of all registers just before the drop. The block factors are listed first; no physical gate is applied by this regrouping.

(3) First suppose none of the registers named by this drop is marked `loan`. They may be allocated temporaries or unmarked inputs. The site is *safe* when its site map factors,

$$\sigma_s = g \otimes W,$$

for a unit vector $g \in \mathcal{H}_v$, the site's *certificate*, and a linear $W : [\![\Omega]\!] \to \mathcal{H}_{\text{rest}}$: the block comes out in the one state g whatever the input.

(4) If the drop also names registers marked `loan`, they must be returned in their unknown original states. Group those input factors as $[\![\Omega]\!] = \mathcal{H}_{\text{loan}} \otimes [\![\Omega']\!]$, the block's loans and the rest, and the site is *safe* when

$$\sigma_s = B \otimes W$$

for an isometry $B : \mathcal{H}_{\text{loan}} \to \mathcal{H}_v$, the site's *block factor*, and a linear $W : [\![\Omega']\!] \to \mathcal{H}_{\text{rest}}$. Clause (3) is the case $\mathcal{H}_{\text{loan}} = \mathbb{C}$, $B = g$.

The quantifier order matters: one g must work for all inputs. Deutsch–Jozsa has $g = |-\rangle$ and $W = X$; the copy program of section 3.2 has no such factorization. At a nonzero site map the unit certificate, or isometric block factor, is determined up to phase. For temporaries, any nonzero vector factor can be normalized by rescaling W. For loans, requiring B to be an isometry is a real

restriction; an arbitrary linear factor cannot be normalized this way. At a zero site map every unit vector certifies a temporary block.

By linearity, basis inputs suffice (lemma 4.3(1)). For loans, check the $\dim \mathcal{H}_{\text{loan}}$ basis maps $y \mapsto \sigma_s(|i\rangle \otimes y)$: the site is safe exactly when each is $b_i \otimes W$ for one and the same W and orthonormal b_i. Asking only that each output be a product is not enough: a site that parks $|0\rangle$ on the block and moves the loan into the rest passes that test and is unsafe: allocate r in $|0\rangle$, swap the loan a with r, then drop a. The site map is $K\ell = |0\rangle_a \otimes \ell_r$. Every output is a product, but no gate on a can recover the unknown ℓ from its fixed state $|0\rangle$. Loan safety requires the loan's information to stay in the block.

An untouched caller register does not change safety. For example, a callee may flip its loan with X and then drop it: its block factor is $B = X$, and another X returns the loan. If the caller kept half of a Bell pair, the first flip changes $(|00\rangle + |11\rangle)/\sqrt{2}$ to $(|10\rangle + |01\rangle)/\sqrt{2}$; the second restores the whole Bell pair. The callee never touches the caller's half. The following proposition says this works for every input correlation, with the same block factor.

Proposition 4.2 (safety is modular). *Let σ_s be the linear map of a site as in definition 4.1. Let $\mathcal{H}_c \neq 0$ be a client register on which the program does nothing. Then*

$$\sigma_s \otimes I_c = B \otimes W' \text{ for some } W' \quad \Longleftrightarrow \quad \sigma_s = B \otimes W \text{ for some } W.$$

The same isometric block factor B works on both sides, including when the client is entangled with the inputs. For a block without loans, read $B = g$.

A caller that first copies another input into a loan violates this restriction: even an otherwise empty callee then has an unsafe drop. A proper block measurement on a loan followed immediately by its drop is also unsafe on each outcome: the projector cannot be rescaled to an isometry.

Two sites are degenerate and the definition decides both without an extra clause. With $\Omega = \cdot$ the site map is scalar multiplication by one fixed vector, and the site is safe exactly when that vector is a product or zero across $\mathcal{H}_v \otimes \mathcal{H}_{\text{rest}}$. With $\mathcal{H}_{\text{rest}} = \mathbb{C}$, nothing else being allocated, a block of temporaries is safe exactly when σ_s has rank at most one, and, without measurement, a loan-only block can be returned by the inverse of its unitary action. *A product at every input* is weaker than safety since the swap of t and v leaves a product at every input and its site is unsafe, having moved the whole input into the block; proposition B.2 (section B) says a map that is a product at every input either fixes the block or fixes the rest. Safety requires a fixed block; the two forms can overlap.

Proposition 4.4(3) describes how gates change the Gram matrix. More generally, any linear operation on the rest preserves an existing certificate, but a projection can make an unsafe site safe on its selected record.

A tuple drop is one site. A fixed Bell pair is safe as $\texttt{drop}\,(a, b)$ and unsafe as either individual drop. More generally, the individual drops are safe exactly when the combined certificate is a full product, under proposition B.3's conditions. For three registers, allocate a, b, c in $|0\rangle$ and apply $H\,a$; $\mathrm{CX}\,a\,b$; $\mathrm{CX}\,a\,c$. The resulting GHZ state $(|000\rangle + |111\rangle)/\sqrt{2}$ is a fixed certificate for $\texttt{drop}\,(a, b, c)$. Each individual register instead has state $I/2$, so no individual drop is safe.

4.2 The factorisation, and the channel

Lemma 4.3 (factorisation, and proportional maps).

(1) Let $\sigma : [\![\Omega]\!] \to \mathcal{H}_v \otimes \mathcal{H}_{\text{rest}}$ be linear and $g \in \mathcal{H}_v$ a unit vector. Then $\sigma(x) \in g \otimes \mathcal{H}_{\text{rest}}$ for every x if and only if $\sigma = g \otimes W$ for $W := (\langle g| \otimes I) \circ \sigma$. Then $\|Wx\| = \|\sigma(x)\|$ for every x, so W is an isometry exactly when σ is, and a contraction — a map that never increases norm — exactly when σ is.

(2) *Let $A, B : X \to Y$ be linear and suppose $Ax \in \mathbb{C}\,Bx$ for every x. Then $A = cB$ for one scalar c.*

For a site map $K : [\![\Omega]\!] \to \mathcal{H}_v \otimes \mathcal{H}_{\text{rest}}$, including a map on a measurement record, put

$$d = \dim \mathcal{H}_v \geq 1, \qquad n = \dim [\![\Omega]\!] \geq 1, \qquad K_i = (\langle i| \otimes I)K, \qquad \Gamma_{ij} = \operatorname{tr}(K_i^\dagger K_j).$$

The Gram matrix Γ records the pairwise inner products of the operators K_i. No isometry hypothesis is needed in the proposition.

Proposition 4.4 (the Gram test).

(1) Fixed block. *At a site without loans this is the safety test: the following are equivalent:*

$$K = g \otimes W \text{ for a unit } g \iff \operatorname{rank}\Gamma \leq 1 \iff \Gamma^2 = (\operatorname{tr}\Gamma)\Gamma.$$

(2) Least block dimension. *The map $\rho \mapsto \sum_i K_i \rho K_i^\dagger$ can be represented using a block of dimension $r \geq 1$ exactly when $r \geq \max(1, \operatorname{rank}\Gamma)$.*

(3) Changing gates. *A unitary on the rest leaves Γ unchanged. A unitary U on the block changes it to $\bar{U}\Gamma U^\top$.*

At input I/n (a uniformly random basis input), the normalized block state is $\Gamma^\top / \operatorname{tr}\Gamma$ when $K \neq 0$. Without loans, safety requires purity, not just input independence (proposition B.1).

At a nonzero site, a safe temporary contributes one independent direction to the Gram matrix. A safe loan qubit must contribute two: its unknown state has two independent basis inputs, which the block must retain. More generally, the required number is the dimension of the loan input. This gives a quick way to reject a loan site, but does not replace the factorization test.

Proposition 4.5 (Gram rank can refute loan safety). *If a nonzero site map K is safe with loans in its block, then $\operatorname{rank}\Gamma = \dim \mathcal{H}_{\text{loan}}$. A different rank therefore refutes loan safety; equality alone does not establish it. The basis-map test in section 4.1 checks the full condition.*

Proof. At a safe site $K = B \otimes W$, with $B^\dagger B = I$ and $W \neq 0$. Its Kraus operators are $K_i = (\langle i| \otimes I)(B \otimes W) = (\langle i|B) \otimes W$, so $K_i^\dagger K_j = (B^\dagger|i\rangle\langle j|B) \otimes (W^\dagger W)$, whose trace is $\langle j|BB^\dagger|i\rangle \cdot \operatorname{tr}(W^\dagger W)$. Collecting the entries,

$$\Gamma = \operatorname{tr}(W^\dagger W)(BB^\dagger)^\top.$$

The scalar is positive and $BB^\dagger$ projects onto the $\dim \mathcal{H}_{\text{loan}}$-dimensional image of B, proving the rank claim. □

Equality is insufficient: the copy map $|i\rangle \mapsto |i\rangle \otimes |i\rangle$ on a loan qubit has $\Gamma = I_2$. Yet its output at $|+\rangle$ is entangled, so it cannot factor as $B \otimes W$. The nonzero hypothesis matters: a zero site map is safe with any isometric block factor, while its Gram rank is zero.

4.3 The criterion, and contextual equivalence

Turning a certificate back into a program needs one fact about isometries, and it is the step a finite gate set later threatens.

Lemma 4.6 (isometry extension). *Let $\mathcal{H}$ be a finite-dimensional complex Hilbert space. Every isometry $V : S \to \mathcal{H}$ out of a subspace $S \subseteq \mathcal{H}$ is the restriction of a unitary of $\mathcal{H}$.*

The proof completes an orthonormal basis (section B). We use it on the input subspace $[\![\Omega]\!] \otimes |0 \cdots 0\rangle$ of $\mathcal{H}_{\text{rest}}$; temporary blocks contain no input registers. The ring version appears in section 6.

Definition 4.7 (weakening witness). A *weakening witness* at a site is a program $\mathrm{sep}(R, W)$ of the shape

`new` the block in ψ ; R on the block ; `new` other temporaries in $|0\rangle$; W on the rest ; `drop` the block

— *prepare the temporary once, compute on everything else, drop.* The drop is a weakening, of a register no gate after R names. Its open form $\mathrm{sep}^\circ(R, W)$ is the same text read before its final `drop`; the two denote the same map, a `drop` moving no vector, and the open form returns every register.

Deutsch–Jozsa uses a joint gate yet has a witness of this shape (section 4.4).

We compare these maps in every *closed context*. A term context $\mathcal{C}$ is a surface term with one typed hole: $\mathcal{C}\{N\}$ is closed and has type A' whenever N has the hole's context and type. It supplies inputs using allocations and gates, and may act on the hole's outputs. A surface block lets the context bind the returned registers and apply more gates to them; a tail-position core hole alone does not.

For example, Z changes $\alpha|0\rangle + \beta|1\rangle$ to $\alpha|0\rangle - \beta|1\rangle$. Direct basis measurement sees the same weights $|\alpha|^2, |\beta|^2$ before and after this change. A context can detect it by preparing $|+\rangle$ and applying H after the hole: $HI|+\rangle = |0\rangle$, whereas $HZ|+\rangle = |1\rangle$. Thus observing an output includes choosing how to measure it.

The *answer* of a closed program $\cdot \vdash P : A'$ is the weight of each standard-basis outcome of its output register, $b \mapsto \|(I \otimes \langle e_b|)[\![P]\!]\|^2$, and $M \cong M'$, for two terms of one judgement, when every closed context gives the two the same answer, compared exactly. This is contextual equivalence with outcome weights as answers [16, Chapters 46–47]. For a closed measurement-free program these weights form a probability distribution. Contextual equivalence requires agreement after every allowed preparation and output operation, not just direct basis measurement.

The relation is a *congruence*: replacing a term by an equivalent one preserves equivalence of the larger program, since further observations are themselves contexts. It respects answers of closed programs. It is the *coarsest* such relation, identifying the most pairs: any congruence respecting answers can equate only terms that all closed contexts fail to distinguish.

Remark 4.8 (which program a context compares). Theorem 4.10 compares prefixes with every register exposed. Earlier drops must be deleted, because a context cannot read a hidden register. To see why, compare these closed programs:

```
new a : [2] @ |0>;          new a : [2] @ |0>;
new b : [2] @ |0>;          new b : [2] @ |0>;
H a; CX a b;
drop a; drop b; return ()   drop a; drop b; return ()
```

Both return only (), with weight one, so no context distinguishes them. Yet the first drop is unsafe on the left, where a is half of a Bell pair, and safe on the right, where a is in $|0\rangle$. Returning (a, b) before either drop makes the difference observable. The criterion therefore compares those exposed prefixes, rather than programs whose drops have already hidden the information it needs.

Setting. $\mathbb{K} = \mathbb{C}$ and $\mathcal{G}$ is every unitary. M is measurement-free and s a site of it whose block is a tuple of temporaries, allocated in the unit state $\psi \in \mathcal{H}_\nu$. $M|_s$ is the prefix of M before s with every `drop` deleted (remark 4.8), so that $[\![M|_s]\!] = \sigma_s$; it returns every register, as $\mathrm{sep}^\circ(R, W)$ does.

Lemma 4.9 (equal answers, proportional maps). *Let two prefixes with every* `drop` *deleted have the same context and type, so that their denotations $\sigma, \sigma' : [\![\Omega]\!] \to \mathcal{H}$ are linear maps into one space. If no closed context tells them apart, then $\sigma' = c\,\sigma$ for a scalar c, of modulus one once $\sigma \neq 0$. Conversely, no closed context tells apart two prefixes with $\sigma' = c\,\sigma$ and $|c| = 1$.*

Theorem 4.10 (the criterion, and contextual equivalence). *In the Setting, the following are equivalent.*

(1) s is safe.

(2) $\sigma_s = g \otimes W$ for a unit $g \in \mathcal{H}_v$ and an isometry $W : [\![\Omega]\!] \to \mathcal{H}_{\text{rest}}$.

(3) Some weakening witness $\text{sep}(R, W)$ has denotation σ_s.

(4) No closed context tells $M|_s$ from some $\text{sep}^\circ(R, W)$.

Two prefixes no closed context tells apart have the same certificates.

Already the contexts that prepare one input and read one observable decide clause (4), and running the prefix twice separates nothing more, two copies carrying the phase c^2.

The witness needs exact gates preparing the certificate and extending the rest isometry; restricted gate sets may lack them (section 6). For an arbitrary linear map K, having a measurement-free witness requires $K = g \otimes W$ together with $\|Kv\| = \|v\|$ for every v, and the clause cannot be dropped: at $d = m = 1$ every map is $g \otimes W$, and $v \mapsto 2v$ is no typed program's denotation.

Read as a predicate on the site maps of measurement-free programs, safety is the least one that holds of every restoration and is unchanged by a gate on the block: a site is safe exactly when its map is $(V \otimes I)(\psi \otimes W)$ for a unitary V on the block and an isometry W into the rest. Restoration, purity of the block's own state, Liu, Zhou and Meng's validity and Bona's body condition each differ from it on a one-line program (section 9).

4.4 Worked: Deutsch–Jozsa

For Deutsch–Jozsa (section 3.1), take $f : \{0,1\} \to \{0,1\}$, so that $U_f|t, j\rangle = |t \oplus f(j), j\rangle$ and $D_f = \text{diag}((-1)^{f(0)}, (-1)^{f(1)})$. At an *arbitrary* input $\sum_j c_j|j\rangle$ to U_f—not only the intended one—the state after it is $|-\rangle \otimes \left(\sum_j (-1)^{f(j)} c_j |j\rangle\right)$, with the temporary first as throughout this section. Thus the site map is $|-\rangle \otimes HD_fH$, with $HD_fH = \pm I$ for constant f and $\pm X$ for balanced f. The drop is safe by theorem 4.10, although its certificate is not the allocation state $|0\rangle$.

4.5 The reclaiming gate

At a nonzero safe temporary site, $\Gamma^\top / \operatorname{tr}\Gamma = gg^\dagger$ determines the certificate ray. We choose a representative to construct its reclaiming gate.

We fix the representative by its overlap with the allocation state, using the computational basis only to break a tie at zero overlap. A basis-independent choice is impossible: multiplication by $c \neq 1$ fixes a ray but moves every nonzero vector on it. Preparation may need more than a permutation, which cannot take $|0\rangle$ to $|+\rangle$.

Setting. The temporary site s is safe with certificate $[g]$ and allocation state ψ. Use the representative $\tilde{g}$ fixed in section 3.6, and put $w = \psi - \tilde{g}$.

Proposition 4.11 (reclaiming temporaries). *In the Setting above, define*

$$R^\psi = \begin{cases} I, & w = 0, \\ I - 2ww^\dagger / \|w\|^2, & w \neq 0. \end{cases}$$

(1) Exchange. *R^ψ is the unique linear map that exchanges $\tilde{g}$ and ψ and fixes their common orthogonal complement. It is a unitary involution.*

(2) No gate needed. *$R^\psi = I$ exactly when $\tilde{g} = \psi$, equivalently when g lies on the allocation ray.*

(3) Restoration. *At every input, applying R^{ψ} to the block puts it in ψ and leaves the reduced state of the rest unchanged.*

A dropped block may contain both loans and temporaries. A loan must be returned in the state the caller supplied; a temporary must be returned to its declared allocation state.

At a safe site, the block factor $B : \mathcal{H}_{\text{loan}} \to \mathcal{H}_{\nu}$ of definition 4.1(4) describes where the program has put the loans' information. For a loan input ℓ, the block contains $B\ell$. Let J describe the desired result: $J\ell$ places the unchanged loans beside the temporaries in their allocation state ψ. If loans precede temporaries in the tuple, $J\ell = \ell \otimes \psi$; otherwise these factors are placed in the tuple's order.

Proposition 4.12 (returning loans). *For the safe block above, there is a unitary T on the block such that*

$$T(B\ell) = J\ell \qquad \text{for every loan input } \ell.$$

Thus the same gate restores every possible loan state and every temporary's allocation state. It also preserves the loans' correlations with registers kept by the caller.

Only the action of T on the image of B is prescribed. Different unitaries may extend that action to the rest of the block space. If the block contains only loans, B is square and unitary: $J = I$ and $T = B^{\dagger}$. In this case T is the identity up to phase exactly when B is.

Here *reclaiming a temporary block* means restoring its declared allocation state by a unitary on that block alone, at every input. Once restored, the block can serve a later allocation without changing the rest of the machine. The gate need not reverse the computation that produced the block.

This meaning gives the converse as well. If one fixed unitary T restores the block to ψ at every input, the resulting site map is $(T \otimes I)\sigma_s = \psi \otimes W$ for some W. Applying $T^{\dagger}$ gives $\sigma_s = (T^{\dagger}\psi) \otimes W$, so the original site is safe. This is a guarantee for arbitrary later use. A particular continuation may happen not to observe an unsafe reuse.

For the program in fig. 5, $B\ell = X\ell \otimes |+\rangle$ and $J\ell = \ell \otimes |0\rangle$. The gate $T = X \otimes H$ satisfies $TB = J$ because $X^2 = I$ and $H|+\rangle = |0\rangle$. If the caller keeps a qubit r entangled with the loan, the same equality holds with I_r tensored on: the whole loan–caller state is restored. No knowledge of α or β is needed to choose T.

The phase convention matters for exact vector equality: the reflection in the hyperplane orthogonal to $\psi - g$ carries g to ψ only when $\langle \psi, g \rangle$ is real, and at $\langle \psi, g \rangle = 0$ the representatives $|1\rangle$ and $i|1\rangle$ give X and Y, respectively, when $\psi = |0\rangle$. Every phase is allowed by the real-overlap condition in this orthogonal case; our coordinate tie-break selects $|1\rangle$ and X.

Proposition 4.13 (reuse is sound). *Extend the machine of fig. 3 with a pool: at a safe site, Step-Drop-Reclaim applies R^{s} to the block and moves its registers to the pool instead of hiding them (loans are returned by the gate of proposition 4.12 and are never pooled); Step-New-Reuse serves a* new *$t : [d] @ \psi$ from a pooled register standing in ψ instead of allocating a fresh one.*

(1) *Every run of the pooled machine is simulated by a run of the machine of fig. 3: the same records, the same weight at every input, and the same reduced state on the returned registers, on every record.*

(2) *The pooled machine allocates at most as many registers, and strictly fewer whenever a reclaimed register is served.*

(3) *If reclamation is allowed at an unsafe site, reuse can be unsound. Extend program (b) by allocating a fresh qubit and returning it with ν. If that allocation instead reuses the dropped block, no gate on the block alone makes the returned joint state agree at every input.*

An unsafe block can be reused for a restricted scope that applies only block-local unitaries and then drops it: the rest's reduced state is unchanged. Coupling it to another register by one CX can change an answer; lifting it can change record weights.

Restoration is the condition enforced by the fractional types of Chen et al. [7]. In their reversible language, allocating a value also produces a garbage collector specialized to that value; deallocation must supply the matching value again. The corresponding requirement here is that a temporary return to its allocation state. Our criterion asks an earlier question: can one local gate bring it back? Deutsch–Jozsa illustrates the difference. Its temporary ends in $|-\rangle$, so it has not been restored to $|0\rangle$, but its state is fixed and the reclaiming gate returns it to $|0\rangle$.

On the artifact's catalogue (`dqt list`), the allocation-state requirement accepts four of the twelve programs whose drops are safe. Among the eight refusals are both Deutsch–Jozsa oracles and both Bernstein–Vazirani keys: all four leave the temporary on $|-\rangle$.

5 Measurement, and what it changes

Different measurement records can give different verdicts and reclaiming gates. We add block measurements and classical branching to express this. A *block measurement* tests whether a register's level lies in a chosen set of levels, and that set is the block it names, a different sense from the registers a drop names.

5.1 Two programs

The first is program (c) of example 1.1, written out on a qubit.

```
prog c (q : [2]) : [2] {
  new t : [2] @ |0>;
  H q;  CX q t;
  p = lift[{0}] t;
  V t q;
  drop t;
  return q
}
```

With V from example 1.1 and the temporary first, the two site maps send $x_0|0\rangle + x_1|1\rangle$ to

$$\tfrac{x_0+x_1}{\sqrt{2}}|0\rangle \otimes |0\rangle, \qquad \tfrac{x_0-x_1}{2}(|01\rangle + |10\rangle).$$

The first map has certificate $|0\rangle$; the second has none. The verdict changes with p, though the program never reads it (theorem 5.6).

The second example is the artifact's `teleport.s`.

```
prog teleport (b : [2]) : [2] {
  new m : [2] @ |0>; H m;
  new a : [2] @ |0>;
  H a; CX a b;
  CX m a; H m;
  z = lift[{0}] m;
  w = lift[{0}] a;
  if w {} else { X b; }
  if z {} else { Z b; }
  drop m; drop a;
  return b
}
```

At input $b = |0\rangle$, this prepares a Bell pair on a, b and teleports the fixed message $|+\rangle$ from m to b. Safety also holds at other inputs to b, where a, b need not be a Bell pair. Each lift fixes its measured qubit at $|0\rangle$ when the recorded bit is 1, and at $|1\rangle$ when it is 0. The corrections touch only b. Both drops are therefore safe on every record. Their recorded bits select between the precomputed reclaiming gates I and X (proposition 5.9).

5.2 Measurement, and its record

Program (c) needs a language in which a measured outcome becomes a classical parameter. From here on a program may contain the two forms of fig. 2's second block, and a *parameter context* Π, a finite set of parameter names, joins the register context in every judgement: $\Pi; \Omega \vdash M : A$, with the rules of definition 3.1 carrying Π unchanged through their premises.

Two rules are added to definition 3.1. The first binds a parameter and keeps the register — under its own name, so a lift binds no register at all; the second is the only place a parameter is read.

$$\frac{S \subseteq [d] \text{ nonempty, proper} \quad \Pi, p;\ \Omega, x : [d] \vdash M \ :\ A}{\Pi;\ \Omega, x : [d] \vdash p = \mathtt{lift}_S\, x;\ M \ :\ A}\ \textsc{Lift}$$

$$\frac{\Pi \vdash e \text{ bool} \quad \Pi;\ \Omega \vdash M_1 \ :\ A \quad \Pi;\ \Omega \vdash M_2 \ :\ A}{\Pi;\ \Omega \vdash \mathtt{if}\ e\{M_1\}\ \mathtt{else}\ \{M_2\} \ :\ A}\ \textsc{If}$$

The nonempty, proper subset makes this a nontrivial test; it does not ensure both outcomes are reachable. We add two rules to fig. 3: Step-Lift, in two labelled instances,

$$\langle \Delta, \sigma \parallel p = \mathtt{lift}_S\, x;\ M\rangle \xrightarrow{b} \langle \Delta, (P_S^b)_x\, \sigma \parallel M[b/p]\rangle \qquad (b \in \{0,1\}),$$

with $P_S^1 = P_S$ the projection of $[\![[d]]\!]$ onto the levels in S, $P_S^0 = I - P_S$, and $(P_S^b)_x$ that projection applied to the factor x names. Thus $b = 1$ means membership in S, not necessarily measured value 1. The rule Step-If replaces `if` $e\{M_1\}$ `else` $\{M_2\}$ by M_1 or M_2 according to the value of the closed guard e.

Lift makes a measurement outcome available to classical control inside the program. Proto-Quipper-Dyn's dynamic lifting additionally transfers such a result to the circuit-generation stage [12]; our language has no separate host stage. Measurement and classical feed-forward also appear in Proto-Quipper-L [27]. Measuring and then discarding a qubit is the derived form `lift` followed by `drop`, safe on both records of a single-level test. For a register of higher dimension, a surviving block with several levels can remain entangled with the rest, making that drop unsafe. Dropping a quantum register does not erase its recorded classical bits. Parameters are classical, so Π is a set and not a linear context, and p may go unread. A guard e is a bit constant, a parameter, a negation, a conjunction or a disjunction of guards, and $\Pi \vdash e$ bool holds exactly when every parameter of e is in Π; a closed guard evaluates to a bit. If asks both branches for the same register context and the same type; within that a branch may allocate, lift and drop registers of its own. Step-Lift is the only rule that can change the norm of σ. An instance preserves that norm exactly when the other's state is zero.

A lift gives two possible next configurations, one for each recorded bit. Repeating this at every lift gives a *tree of runs*. A run follows one path from the initial configuration to a return; its *measurement record* lists the lift outcomes in their order. Once those bits are fixed, all classical guards and all later steps on that path are determined. This is the trajectory viewpoint of quantum measurement [44].

We keep the state on each branch unnormalised. Its squared norm is the probability of reaching that branch, so the state carries both the quantum information and the branch's weight. Dividing by its norm would lose that weight. When the measurement record is ignored, we add the branches' unnormalised density operators, not their state vectors [36].

A measurement tree, step by step. In fig. 8, q holds an arbitrary unit input x. The new temporary t starts in $|0\rangle$, and `H t` prepares $|+\rangle$. The first lift records $p = 1$ when t is $|0\rangle$ and $p = 0$ when it is $|1\rangle$. The conditional flips q only in the second case. It never changes t, so the second test of t must agree with the first. The mismatching paths are drawn too: their states are zero.

5.3 One matrix per record

Because the record fixes every choice, a run is again a composite of linear maps. Write $E_o(M)$, the *per-record map* of M along o, for the composite the run performs: a gate contributes its unitary, a

```
prog twice (q : [2]) : [2] {
  new t : [2] @ |0>;
  H t;
  p = lift[{0}] t;
  if p {} else { X q; }
  r = lift[{0}] t;
  drop t; return q
}
```

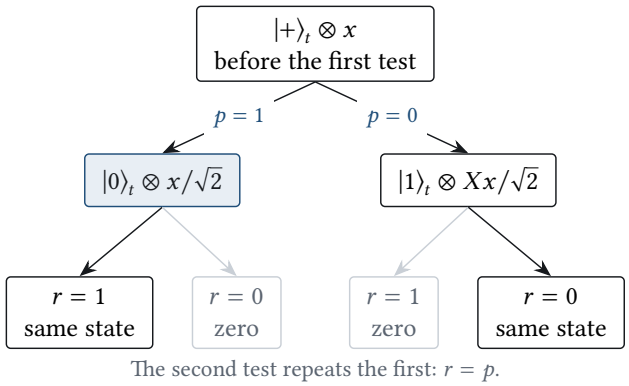


Fig. 8. Read the code and the tree together. Each edge records one lift outcome; the boxes show the state with t first. The first measurement has two outcomes of weight $1/2$. The second repeats it without changing t, so only records 11 and 00 have nonzero weight. The conditional changes q, not the later outcome of measuring t.

`new` adjoins ψ, `drop` contributes the identity, a lift contributes the block projection its bit selects, and an `if` contributes nothing but the choice of branch. Each is a map

$$E_o(M) \;:\; [\![\Omega]\!] \longrightarrow \mathcal{H}^o_{\mathrm{hid}} \otimes [\![A]\!], \qquad \sum_o E_o(M)^\dagger E_o(M) \;=\; I,$$

the hidden factor carrying the superscript because branches may allocate and drop differently. The equation holds by induction along the tree, a lift's two projections being complementary and orthogonal, and every other step preserving norm. Each $E_o(M)$ is a composite of isometries and projections, hence a contraction; the family is complete. The *weight* of the run at a unit input x is $\|E_o(M)x\|^2$, and the displayed equation says the weights of the reachable records sum to one. A record with $E_o(M) = 0$ is *unreachable*, of weight zero at every input. Sites after a lift that makes the state map zero have zero site maps and are safe with every certificate. Sites before that lift retain the verdict of their own prefix; the complete record being unreachable does not make those earlier maps zero (fig. 9).

The matrices of the tree. For `twice`, $\Omega = (q : [2])$ and the return type is $[2]$. After allocation the machine keeps $\Delta = (q : [2], t : [2])$ to the end, even after t is dropped. The parameter context Π gains p at the first lift and r at the second; both are classical bits. On record 11, the first projection keeps $|0\rangle_t \otimes x/\sqrt{2}$, the conditional does nothing, and the second projection keeps the same vector. The drop changes only the names still in scope. Thus the final hidden register is t and the returned register is q.

Writing t first in the denotation, the four maps are

$$E_{11} = \frac{1}{\sqrt{2}}|0\rangle \otimes I, \qquad E_{00} = \frac{1}{\sqrt{2}}|1\rangle \otimes X, \qquad E_{10} = E_{01} = 0.$$

For example, with $P_1 = |0\rangle\langle 0|$ and $P_0 = |1\rangle\langle 1|$ on t, the whole composition is

$$E_{pr} = (P_r \otimes I)(I \otimes X^{1-p})(P_p \otimes I)(H \otimes I)(|0\rangle \otimes I).$$

Read this right to left: allocate, apply H, take the first outcome, execute the conditional, then take the second outcome. Each nonzero map satisfies $E_o^\dagger E_o = I/2$, so its weight is $1/2$ at every unit input. The other two maps vanish, and the four effects sum to I. At the drop, the certificates are $|0\rangle$ on 11 and $|1\rangle$ on 00; the two zero maps admit every certificate.

This example has no lifts after its drop. To see why later outcomes cannot repair an earlier verdict, use program (c) instead and append the test in fig. 9. The new register r is in $|0\rangle$, so testing membership in $\{1\}$ makes the $b = 1$ continuations zero. It does not alter the map at the earlier drop of t. In particular, that drop remains unsafe on its $p = 0$ prefix.

Theorem 5.1 (type safety, with the two forms). *Theorem 3.3 holds of the five-rule machine, with two clauses read again and two unchanged.*

(1) Progress *now reads: a well-typed configuration is final, or exactly one rule applies to it, and at Step-Lift both of that rule's labelled instances do — a lift being the only labelled step.*

(2) Termination *now reads along the branch taken: the run whose record is o reaches a final configuration in at most as many steps as that branch has statements, an `if` counting as one, and so in at most $|M|$.*

(3) Preservation *stands word for word, and so does the hidden-register invariant of section 3.3: a lift binds its parameter and leaves its register live under its own name, so no register enters or leaves the context there.*

(4) Adequacy *stands with the run read along its record: the run from $\langle \Omega, x \parallel M \rangle$ whose record is o ends in the state $E_o(M)\, x$.*

Proofs are in section C.

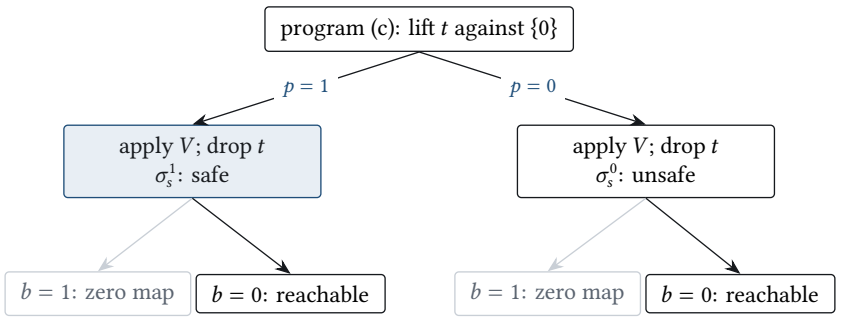


Fig. 9. A later zero-weight outcome cannot change an earlier verdict. After program (c)'s drop, append `new r : [2] @ |0>; b = lift[{1}] r; drop r`. Both $b = 1$ leaves are unreachable; the earlier $p = 0$ site remains unsafe.

5.4 The site, read on one record

Definition 4.1 now applies *on the record o*. A site s is an occurrence of `drop` in the file, and it fires at most once on a run since no step rule duplicates a term, and the copies that the desugaring of section 3.1 makes of a `drop` lie in different arms of an `if`, of which a run enters one. A site inside one arm fires on the records that enter that arm and on no other, one verdict on each.

When it fires on the record o, its site map $\sigma_s^o : [\![\Omega]\!] \to \mathcal{H}_v \otimes \mathcal{H}_{\text{rest}}$ sends x to the machine's state at that step. For a block containing no loans, s is *safe on o* exactly when $\sigma_s^o = g \otimes W_o$ for one unit g; the verdict is still read off the block's Gram matrix, safe exactly when its rank is at most one, with nothing to normalise.

A closed context *tells two prefixes apart on o* when, with each run along o at the hole, it gives them different answers. Lift-free contexts settle this: a context that lifts and branches is linear in its hole along o, so by lemma 4.9 it tells apart no pair the lift-free ones do not.

The safe record of program (c) kills $|-\rangle$ and preserves the norm of $|+\rangle$. Its map is a contraction, not an isometry, so it cannot have a lift-free witness. One additional lift suffices, and is needed exactly when the site map is not isometric. A lift after the site acts on the rest. It may change the final record weights, but cannot change the earlier site map or its verdict. If the site is safe, both outcomes retain its certificate. Their weights sum to the weight before the lift. The verdict is read at the site and not after it, where proposition B.3's Bell witness leaves the block on a basis vector.

Theorem 5.2 (the criterion, one record at a time). *Let s be a site of M whose block is a tuple of temporaries allocated in the unit state ψ, and o a record on which s fires. The following are equivalent.*

(1) s is safe on o.

(2) $\sigma_s^o = g \otimes W_o$ for a unit $g \in \mathcal{H}_v$ and a contraction $W_o : [\![\Omega]\!] \to \mathcal{H}_{\mathsf{rest}}$.

(3) A witness has map σ_s^o on its kept record*, where its fresh qubit a reads 0, after omitting that fixed $|0\rangle_a$ factor. The witness performs three steps: prepare the block in ψ and apply one gate R to it; allocate the other temporaries and a in $|0\rangle$ and apply one gate U to everything except the block; then lift a against $\{0\}$.*

(4) No closed context, and already no lift-free one, tells $M|_s$ on o from some such program on its kept record.

Corollary 5.3 (safety is an observational invariant, one record at a time). *Fix a record o and let two prefixes with every* drop *deleted have the same context and type, and let no lift-free closed context tell them apart on o. Then $\sigma'^o = c\,\sigma^o$ for a scalar c, of modulus one once $\sigma^o \neq 0$; hence their sites are both safe or both unsafe on o, and every certificate of one is a certificate of the other.*

A lift against $\{j\}$ fixes the measured register at $|j\rangle$ on its surviving record; on a qubit, the other record fixes the other basis state. If the register stays untouched until a drop naming it alone, that state certifies the drop. Teleportation uses this case.

5.5 A block test needs only one measured qubit

A test $\mathtt{lift}_S\, x$ asks one yes-or-no question: is the level of x in S? It need not measure which level of x occurred. Instead, a fresh *indicator qubit* stores that one answer and is measured. This preserves superpositions among the levels inside each outcome.

For example, let $x : [4]$ be in $(|0\rangle + |1\rangle + |2\rangle + |3\rangle)/2$ and take $S = \{0, 2\}$. Allocate a in $|0\rangle$ and flip a exactly on levels 0 and 2 of x. The state becomes

$$\frac{|0\rangle + |2\rangle}{2} \otimes |1\rangle_a + \frac{|1\rangle + |3\rangle}{2} \otimes |0\rangle_a.$$

Testing a against $\{1\}$ selects the first sum when $p = 1$ and the second when $p = 0$, each with weight $1/2$. It does not distinguish $|0\rangle$ from $|2\rangle$, or $|1\rangle$ from $|3\rangle$. On either record a is in a known basis state, so it is safe to drop: apply X on record 1 to restore a to $|0\rangle$, and do nothing on record 0.

Remark 5.4 (a block lift is a derived form). Let $S \subseteq [d]$ be nonempty and proper. Define the indicator gate

$$U_S = \sum_{j \in S} |j\rangle\langle j| \otimes X + \sum_{j \notin S} |j\rangle\langle j| \otimes I.$$

It leaves x's level unchanged and flips a fresh qubit a exactly when that level belongs to S. In a signature containing U_S, replace $p = \mathtt{lift}_S\, x;\ M$ by these steps:

(1) Allocate a in $|0\rangle$ and apply U_S to (x, a).

(2) Set $p = \texttt{lift}_{\{1\}}\, a$. Thus $p = 1$ has the same meaning as in the original test: the level of x belongs to S.

(3) Drop a and continue with M.

To check the replacement, write the incoming state as $\sigma = \sum_j |j\rangle_x \otimes \sigma_j$ and put $\tau_1 = \sum_{j\in S} |j\rangle \otimes \sigma_j$ and $\tau_0 = \sum_{j\notin S} |j\rangle \otimes \sigma_j$. Step (1) gives $\tau_1 \otimes |1\rangle_a + \tau_0 \otimes |0\rangle_a$. On outcome b, step (2) keeps $\tau_b \otimes |b\rangle_a = ((P_S^b)_x \sigma) \otimes |b\rangle_a$. This is the original branch state with one extra, fixed factor. The replacement therefore has the same type and per-record map after omitting that factor. Its drop of a is safe, and X^b restores a to $|0\rangle$ (proposition 4.11).

In `shor.s`, order finding for $N = 15$ at base 7 holds its residue in a temporary `w : [16]`, runs the three controlled powers and the inverse transform, and then runs `s = lift[{7}] w;  drop w;`. On the record that keeps $\{7\}$ the temporary is on $|7\rangle$ whatever the counting registers hold, so the drop is safe with certificate $|7\rangle$; and because $|0\rangle$ and $|7\rangle$ are orthogonal, this is a site where proposition 4.11's tie-break decides which reflection is emitted. On the complementary record the temporary is left on a superposition of the other residues, correlated with the counting registers, and the drop is unsafe.

5.6 Why the record matters

For a block without loans, proposition 4.4 decides safety under *every* record by testing each site map, including zero maps. A single boolean cannot report different verdicts on different records.

Definition 5.5 (record-oblivious verdict). For programs with a single drop site, a *record-oblivious verdict* is a function $\mathcal{D}$ from programs to booleans: the measurement record is not an argument. It is *record-exact* on a class C when, for every $c \in C$ and every reachable record o on which its drop fires, $\mathcal{D}(c)$ agrees with the safety of c's drop under o. A program is *outcome-dependent* when it has two reachable records of opposite verdict.

Figure 10 draws the two runs.

Theorem 5.6 (the verdict is record-dependent). *Let c be program (c) of section 5.1.*

(1) c is well typed, and its two records are both reachable, each of weight $\frac{1}{2}$ at every basis input.

(2) Under the record taking 1 *at the lift, the temporary landed in $\{0\}$, and the site map is $\sigma_{\text{in}}(x) = \frac{x_0+x_1}{\sqrt{2}} |0\rangle \otimes |0\rangle$, and the drop is safe, with certificate $|0\rangle$.*

(3) Under the record taking 0 *the site map is $\sigma_{\text{out}}(x) = \frac{x_0-x_1}{2} (|0\rangle \otimes |1\rangle + |1\rangle \otimes |0\rangle)$, and the drop is unsafe.*

One gate already gives two records of opposite verdict: a single two-qubit gate on the temporary and q, then a lift of q. In program (c), the first Hadamard makes both weights $\frac{1}{2}$ at every basis input; the third gate V makes the verdicts differ. At those uniform weights, one gate before a lift cannot produce opposite verdicts.

Corollary 5.7 (no record-oblivious verdict is exact here). *The program of theorem 5.6 admits no record-exact record-oblivious verdict: rejecting it discards the safe run at the record that kept $\{0\}$, and accepting it admits the unsafe run at the other.*

Proof. By theorem 5.6 the two reachable records carry opposite verdicts, and one boolean agrees with at most one of them. □

A drop before all lifts has one verdict and, if safe, one reclaiming gate. Later sites can have constant verdicts but varying certificates: teleportation is safe on every record, yet needs the gate table of section 5.9.

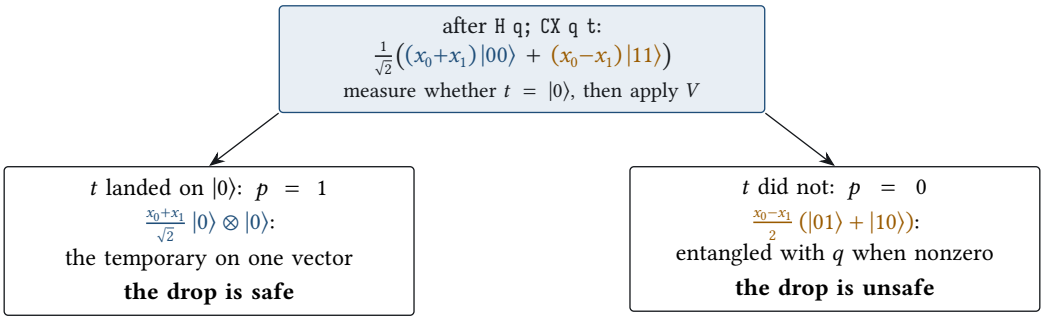


Fig. 10. The two runs of program (c). Look at the two boxes: the same text on both sides, weight one half each at every basis input; on the left the temporary sits on one vector, on the right it is entangled with q when nonzero. Theorem 5.6 is this computation, written out in section C.

The obstruction is not syntactic: a program can contain the witness of theorem 5.6 verbatim and be safe on its only reachable record (section C), so performing a lift is necessary and not sufficient for two verdicts, and every analysis that condemns the pattern is strictly conservative.

5.7 What the summed channel forgets

If a caller ignores the measurement record, it sees only the final quantum output. The summed channel describes that interface: it averages the outputs over all records, with their probabilities. This is enough to predict measurements of the output. It need not tell us whether a particular temporary can be reclaimed on a particular record.

Take a program whose only drop hides a temporary block immediately before the return. Its record map $V_o = E_o(M)$ induces the trace-non-increasing map $\Phi_o(\rho) = \sum_i K_i \rho K_i^\dagger$, where $K_i = (\langle i| \otimes I)V_o$. The trace of $\Phi_o(\rho)$ is the record's weight; $\sum_o \Phi_o$ is the summed channel. By proposition 4.4, the drop is safe on o exactly when Φ_o has a one-Kraus representation, allowing the zero operator for an unreachable record. Keeping the individual maps Φ_o therefore keeps the verdicts. The following pair shows what can be lost when we sum them.

Both programs below prepare t in $|+\rangle$ by applying H to its allocation state $|0\rangle$. In `channelSafe`, the diagonal gate is CZ: it applies Z to q when t is 1. The lift then fixes t at a basis state. In `channelUnsafe`, the lift comes first, but CX subsequently entangles t with q on superposed inputs.

```
prog channelSafe (q : [2]) : [2] {
  new t : [2] @ |0>; H t;
  diag t q (1,1,1,-1);
  p = lift[{0}] t;
  drop t; return q
}
```

```
prog channelUnsafe (q : [2]) : [2] {
  new t : [2] @ |0>; H t;
  p = lift[{0}] t;
  CX q t;
  drop t; return q
}
```

For input $x = x_0|0\rangle + x_1|1\rangle$, their unnormalised states at the drop are as follows, with t first. Every row has weight $1/2$ for unit x.

Record	`channelSafe`: safe	`channelUnsafe`: unsafe
$p = 1$	$\|0\rangle \otimes x/\sqrt{2}$	$(x_0\|00\rangle + x_1\|11\rangle)/\sqrt{2}$
$p = 0$	$\|1\rangle \otimes Zx/\sqrt{2}$	$(x_0\|10\rangle + x_1\|01\rangle)/\sqrt{2}$

The left column has a fixed temporary on each record. The right column does not: at $x = |+\rangle$, each displayed state is entangled. The left program's visible maps are $\rho/2$ and $Z\rho Z/2$; the right program's are both $\mathrm{diag}(\rho)/2$, where diag deletes the off-diagonal entries in the computational basis. Both sums are $\mathrm{diag}(\rho)$. Thus the output channel and even the record probabilities agree, while all the safety verdicts differ.

Proposition 5.8 (the summed channel does not decide). *There are two well-typed programs on a temporary qubit and a visible qubit, each with two reachable records, that have the same summed channel $\Phi_0 + \Phi_1$ and the same weight on every branch at every input, and opposite verdicts on every branch.*

One channel-level test is sound: if the summed channel is conjugation by an isometry, every branch's Kraus operators are proportional to that isometry. Hence every record is safe. The converse fails: the safe program above has different visible actions on its two records, and its summed channel is dephasing, not conjugation by an isometry.

5.8 Deferring measurement changes the site being tested

Deferring a measurement means storing its answer in a quantum register and measuring that register later. The final answers can be unchanged even though the state at an earlier drop is different. For example, the first program below measures t before dropping it. The second stores that answer in a, drops t, and only then measures a.

```
prog measured (q : [2]) : [2] {
  new t : [2] @ |0>; H t;
  p = lift[{0}] t;
  drop t;
  return q
}
```

```
prog deferred (q : [2]) : [2] {
  new t : [2] @ |0>; H t;
  new a : [2] @ |1>; CX t a;
  drop t;
  p = lift[{1}] a;
  drop a; return q
}
```

In `measured`, the two states at `drop t` are $|0\rangle_t \otimes x/\sqrt{2}$ and $|1\rangle_t \otimes x/\sqrt{2}$. Each record fixes t, so both are safe. In `deferred`, a starts at 1, and the controlled negation makes $a = 1$ exactly when $t = 0$. At `drop t`, the state is

$$\frac{|0\rangle_t|1\rangle_a + |1\rangle_t|0\rangle_a}{\sqrt{2}} \otimes x.$$

The temporary t is entangled with a, so this earlier drop is unsafe. Measuring a later gives the same two record weights and the same conditional states of t as the first program. It does not change the verdict at the earlier site. A restoring gate controlled by the coherent record would act on a as well as t; it would not be a gate on the dropped block t alone.

More generally, suppose the branches use the same registers and store their coherent record in orthogonal states $|o\rangle$. Before conditioning on that record, the site map is $\sum_o \sigma_s^o \otimes |o\rangle$. It has a fixed block factor g exactly when every nonzero σ_s^o has that same factor ray $[g]$. The two certificates $|0\rangle$ and $|1\rangle$ in the example are different, so the unconditioned map has no such factor. This comparison concerns outcomes stored before the site; later postselection cannot change that site's verdict.

5.9 Compiling the record-dependent reclaim

The two drops in `teleport.s` are safe on every record, but they do not use one fixed restoring gate. Recall that `z = lift[{0}] m` and `w = lift[{0}] a` return 1 when the measured level is 0. Thus the measured states are $|1 - z\rangle_m$ and $|1 - w\rangle_a$. The compiler can calculate the following table before execution; the recorded bits choose a row during execution.

z	w	state of m	state of a	gate on m	gate on a
1	1	$\|0\rangle$	$\|0\rangle$	I	I
1	0	$\|0\rangle$	$\|1\rangle$	I	X
0	1	$\|1\rangle$	$\|0\rangle$	X	I
0	0	$\|1\rangle$	$\|1\rangle$	X	X

Each row restores both temporaries to $|0\rangle$ and leaves the output register b unchanged. At the advertised input $b = |0\rangle$, every row has weight $1/4$ and the corrected output is $|+\rangle$. For other inputs to b, the weights can differ; the restoring gates in the table still apply.

Proposition 5.9 (the record chooses the reclaiming gate). *Let s fire on record o with $\sigma_s^o \neq 0$, safe there with certificate $[g_o]$ and block allocated in ψ, and let R_o be the reclaiming gate of proposition 4.11 for that certificate.*

(1) g_o and R_o depend on o only through the bits o takes at the lifts that fire before s: two records agreeing on those have the same certificate and the same reclaiming gate.

(2) One gate cannot serve all records: some well-typed M has a site whose reclaiming gate is I on one record and X on the other, whether or not an `if` of M reads the bit that decides.

The table has at most 2^{ℓ_s} rows, where ℓ_s bounds the number of lifts on any path to the site. A smaller bound applies to a fixed sequence of block tests on unchanged registers. Group basis levels that give the same answers to every test: each such group, or *cell*, permits just one record. For instance, the repeated test in fig. 8 has only the two cells $\{0\}$ and $\{1\}$, so it has at most two reachable records. If the sequence tests register r of dimension d_r exactly n_r times, its reachable records are at most the nonempty cells in the common refinement of the tests, and there are at most $\prod_r \min(2^{n_r}, d_r)$ such cells. Intervening gates can invalidate this bound: H; measure; H; measure on one qubit reaches all four records, whereas the bound for unchanged registers would give two.

A qubit lift against $\{j\}$ emits `c[k] = measure q`; the source bit is 1 exactly when `c[k] == j`. A *condition* is the classical test guarding the chosen gate. For example, reclaiming m in the table requires X exactly when the measured bit for m is 1, or equivalently when $z = 0$. It requires no test of w. Larger blocks use the indicator qubit of remark 5.4.

The emitter uses only bits recorded before the drop. It preserves the source's enclosing branches, so it can repeat the same gate under different conditions: in `teleport.s`, the X on m appears in both branches for w. It does not in general minimize the set of bits mentioned in a condition. The mathematical table may therefore be simpler than the emitted control structure. Acting on the hidden block alone, a reclaiming gate commutes with every later source step and changes no later outcome's weight. A backend may therefore move it past a later measurement.

Theorem 5.10 (compilation, one record at a time). *Let M be in the compilation fragment, with every lifted register of power-of-two dimension. Fix a record o. List the safe sites that fire on this record as $s_1, \dots, s_r$, in execution order. Assume that each site's certificate or loan block factor is emittable, and write $R_o^{s_j}$ for its ideal reclaiming gate.*

Let C_o be the corresponding branch map of the emitted circuit. When comparing maps, omit any additional indicator qubits that the emitter returns to $|0\rangle$. Order the remaining hidden factors by site and drop-tuple order, as in theorem 3.5. Then:

(1) The compiled branch is the source branch followed by the chosen reclaiming gates:

$$C_o = z_o\big(R_o^{s_1} \otimes \cdots \otimes R_o^{s_r} \otimes \mathrm{id}\big)E_o(M), \qquad |z_o| = 1.$$

The phase z_o is independent of every input, including loan states.

(2) *The branch weight and the reduced output state agree with those of $E_o(M)$ for every input. If no gate is emitted, the entire branch maps agree up to z_o.*

(3) *Each reclaimed temporary is in its declared allocation state, and each reclaimed loan is in its original state. The phase in clause (1) changes neither of these states.*

The ideal-translation and implementation distinction of theorem 3.5 applies here too.

6 The gate ring

Safety is a factorization over $\mathbb{C}$. A coefficient ring restricts whether the certificate and witness can be expressed using the program's scalars. For example, no unit vector over $\mathbb{Z}[1/2]$ lies on the ray of $|-\rangle$. No gate over that ring can reclaim $|-\rangle$ to $|0\rangle$, even with a clean ancilla returned clean. An ancillary register allowed to remain changed gives a different result: $(H \otimes H)(X \otimes I)$ prepares $|-\rangle \otimes |+\rangle$ from $|00\rangle$ and has dyadic entries. Likewise, allocate t at $|0\rangle$, apply the single dyadic gate $H \otimes H$ to (t, v), and drop t. The certificate is $|+\rangle$, so this safe program has no weakening witness over $\mathbb{Z}[1/2]$. Individual H gates are not gates over that ring.

Setting. A *gate ring* $\mathcal{A}$ is a subring of $\mathbb{C}$ closed under complex conjugation. This section takes the two parameters of section 3.1 to be $\mathbb{K} = \mathcal{A}$ and $\mathcal{G}$ the unitaries with entries in $\mathcal{A}$, and calls a program of that language a *program over $\mathcal{A}$*: its gates *and its allocation vectors* are over $\mathcal{A}$, since a `new` puts ψ on a register as surely as a gate would. A clean $|i\rangle$ is over every ring. Say $\mathcal{A}$ *splits* a rank-one projection P when $P = zz^\dagger$ for a z over $\mathcal{A}$. A certificate is a ray $[g]$ and $gg^\dagger$ is that ray's projection, so $\mathcal{A}$ splits $gg^\dagger$ exactly when $[g]$ carries a unit vector over $\mathcal{A}$. A *unit of the ring* is a scalar whose inverse also lies in it.

A *column lemma* completes any unit vector over the ring to a unitary over it. The five rings of fact D.1 have this property in every dimension. From it, orthonormal vectors over $\mathcal{A}$ complete to a unitary over $\mathcal{A}$ by induction on the columns, and with it the criterion relativises at every temporary site of a measurement-free program. Splitting and the column lemma are different properties and buy different halves: the column lemma is what a witness over the ring needs (theorem 6.1), splitting is what puts a certificate over the ring (theorem 6.2), and $\mathbb{Z}[1/\sqrt{2}]$ has a column lemma but does not split every rank-one projection (proposition 6.3). The proofs of this section and the next are in section D.

Theorem 6.1 (the criterion over a gate ring). *Let $\mathcal{A}$ be a gate ring, M a measurement-free program over $\mathcal{A}$, and s a site of M whose block is a tuple of temporaries allocated in $\psi \in \mathcal{A}^d$.*

(1) *If some weakening witness $\mathrm{sep}(R, W)$ with R over $\mathcal{A}$ has denotation σ_s, then s is safe with a certificate carrying the unit vector $R\psi$ over $\mathcal{A}$, whatever W is.*

(2) *If $\mathcal{A}$ has a column lemma (fact D.1) and s is safe with a certificate carrying a unit vector over $\mathcal{A}$, then some weakening witness over $\mathcal{A}$ has denotation σ_s.*

6.1 Clifford+T pays nothing

Theorem 6.2 (the Clifford+T coefficient ring splits). *Let $\mathcal{R} = \mathbb{Z}[1/\sqrt{2}, i]$ and $d \geq 1$.*

(1) *Every rank-one projection on $\mathbb{C}^d$ with entries in $\mathcal{R}$ is $zz^\dagger$ for a vector z over $\mathcal{R}$.*

(2) *For unit g, the ray $[g]$ carries a unit vector over $\mathcal{R}$ exactly when its projection $gg^\dagger$ has entries in $\mathcal{R}$.*
At a safe temporary site over $\mathcal{R}$, suppose the surviving weight at some basis input is a unit of the ring. The certificate then carries a unit vector over $\mathcal{R}$.

A measurement-free temporary site over $\mathcal{R}$ has a witness over $\mathcal{R}$ exactly when it is safe: its record has weight 1.

On a record whose surviving weight at some basis input is a unit of $\mathcal{R}$, clause (2) puts a unit vector over $\mathcal{R}$ on the certificate, and fact D.1 gives a unitary over $\mathcal{R}$ carrying it to the allocation state, in every block dimension. The reclaiming gate can then be searched for inside the program's own gate ring.

At a measurement-free site over Clifford+T, the side condition of theorem 6.1 is therefore automatic. This guarantees a gate over the ring, not a cheap circuit. A signed Pauli string is a tensor product of I, X, Y, Z with sign ± 1. A Clifford gate preserves these strings under conjugation. The one-qubit stabilizer states are the eigenstates of X, Y and Z. For such a one-qubit certificate a reclaiming gate is a Clifford (from $|0\rangle$: the identity, or the reflections X, H, $(Z - X)/\sqrt{2}$ and $(Z \pm Y)/\sqrt{2}$). Reclaiming a nonstabilizer certificate to a computational-basis allocation requires a non-Clifford gate, hence at least one T in a Clifford+T circuit. A single-qubit Clifford carries $|0\rangle$ to a state whose first entry has squared modulus 0, $\frac{1}{2}$ or 1. The certificate $\left(\frac{1}{2}, \frac{1+\omega-i}{2}\right)$, $\omega = e^{i\pi/4}$, has $\frac{1}{4}$. The theorem is about the ring $\mathcal{R}$, the one single-qubit Clifford+T operators live in [22]; the way back to a word over a repertoire is exact synthesis, whose multiqubit results permit a clean ancilla [1, 13, 22] that a witness reusing the site's own allocation does not have, and no finite-repertoire corollary is inferred here.

6.2 Two rings below it do not

Proposition 6.3 (the restricted rings). *Over $\mathbb{Z}[1/2, i]$ and over $\mathbb{Z}[i/\sqrt{2}]$, every rank-one projection with entries in the ring, in every dimension, is $zz^\dagger$ for a z over the ring. Over $\mathbb{Z}[1/2]$ the projection onto $|+\rangle$ is not, and over $\mathbb{Z}[1/\sqrt{2}]$ the projection onto $\cos\frac{\pi}{8}|0\rangle + \sin\frac{\pi}{8}|1\rangle$ is not.*

Allocating over $\mathcal{A}$ rescues neither projection: both rays carry no unit vector over the ring. For a qubit block the first projection is the whole story: the only unit vectors of $\mathbb{C}^2$ over $\mathbb{Z}[1/2]$ are $\pm|0\rangle$ and $\pm|1\rangle$, because a unit vector over $\mathbb{Z}[1/2]$ clears its denominators to a pair of integers p, q with $p^2 + q^2 = 4^m$, and for $m \geq 1$ that forces both to be even: a square is 0 or 1 modulo 4, so a sum of two squares divisible by 4 has both summands divisible by 4. Dividing out a factor of two and repeating leaves $p^2 + q^2 = 1$. So a qubit projection over $\mathbb{Z}[1/2]$ splits exactly when it is diagonal — and so up to dimension three; from four on $(1, 1, 1, 1)/2$ is a unit vector over $\mathbb{Z}[1/2]$, while over $\mathbb{Z}[1/2, i]$ every qubit projection splits.

One gate turns the second projection into a program. With *rot* the rotation of a qubit by $\pi/8$, the gate $rot \otimes rot$ is over $\mathbb{Z}[1/\sqrt{2}]$ although *rot* is not. Applied to a temporary allocated at $|0\rangle$ and a visible qubit, it leaves the temporary on the magic state (section D): the site is safe, with a witness over $\mathbb{Z}[1/\sqrt{2}, i]$ and none over $\mathbb{Z}[1/\sqrt{2}]$ — the same program and certificate, but no reclaiming gate over the smaller ring.

The necessity direction of theorem 6.1 requires no column lemma: a witness over $\mathcal{A}$ supplies the unit certificate $R\psi$ over $\mathcal{A}$. Sufficiency requires completing unit vectors to unitaries over $\mathcal{A}$. We do not settle that problem for arbitrary gate rings in dimension three and above. In dimension two, $\left(\begin{smallmatrix} \bar{v}_0 & \bar{v}_1 \\ -v_1 & v_0 \end{smallmatrix}\right)$ is a unitary over $\mathcal{A}$ sending a unit vector v to e_0. Applying this construction to the certificate and allocation vectors gives a reclaiming gate over $\mathcal{A}$ for a qubit block.

7 Computing the verdict, and its price

The Kraus operators computed in sections 2 and 4.4 give three Gram matrices:

$$\Gamma_{(a)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \qquad \Gamma_{(b)} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \qquad \Gamma_{\mathrm{dj}} = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}.$$

Dividing each transpose by its trace gives the block states $|+\rangle\langle+|$, $I/2$ and $|-\rangle\langle-|$ at the maximally mixed input. Here the matrices are real; in general the transpose matters. Only the first and third are projections, giving the verdicts and certificates of sections 2 and 4.4.

On a record, $\operatorname{tr}\Gamma/n$ is its probability at the maximally mixed input, or its average probability over basis inputs. We now price proposition 4.4's test on a program description.

7.1 One pass over the program

Planned A single pass that carries stabilizer generators along the program and reads the verdict off the ones supported on the block, so that a Clifford site is decided without building its map; and the gates that carry a generator outside the representation, on which the pass gives no answer. Being written; nothing here is claimed.

7.2 On classical programs, deciding is coNP-complete

Theorem 7.1 (the classical price). *Deciding whether a given site whose block contains no loans is safe on a given record is coNP-complete, for programs whose gates are drawn from* $\{X, \text{CNOT}, \text{Toffoli}\}$ *on qubit registers and whose allocations are clean. Hardness holds already for measurement-free programs with one site.*

Corollary 7.2 (no sound polynomial rule is complete). *No polynomial-time procedure is both sound and complete for the safety of a site on a record of a program over* $\{X, \text{CNOT}, \text{Toffoli}\}$, *unless* $\text{P} = \text{NP}$.

Proof. Such a procedure would decide a coNP-complete problem in polynomial time. □

A classical run computes one basis input in linear time; safety quantifies over all inputs. The reduction uses constancy of one output, related to Jordan's strong-equivalence problem [21].

Toffoli is doing the work in that reduction. Without it, a measurement-free program over $\{X, \text{CNOT}\}$ with clean allocations acts on the bits by an affine map over $\mathbb{F}_2$, the field with two elements, because each of its gates does: CNOT replaces a bit y by $y \oplus x$, which is linear in the bits, X replaces y by $y \oplus 1$, which adds a constant, and a clean allocation adjoins the constant 0. A composite of affine maps is affine, so on the n input bits x the program computes $x \mapsto Ax + b$, with a matrix A over $\mathbb{F}_2$ and a vector b, both fixed by the program text. Keep only the rows that belong to the dropped block — the registers that the `drop` names — and the block's own final value is $Lx + c$, where L is those rows of A and c those rows of b.

Safety asks the block to end on one fixed basis string whatever the input, so it asks $Lx + c$ not to vary with x: the drop is safe exactly when $L = 0$. The constant c is left free, a block sitting on a fixed basis string being in a fixed pure state, which is what a certificate is. Testing $L = 0$ takes $n + 1$ runs. Run the zero input, whose output is c; then run each basis bit e_i, whose output is $Le_i + c$, the ith column of L added to that baseline. One run therefore reads one column, and agreement with the baseline on all n of them is $L = 0$.

On a fixed record the lifts are already resolved, so the run is again a fixed sequence of gates and the block's value is again affine, say $L_o x + c_o$. What changes is which inputs it is asked about. Write S_o for the set of inputs that survive the record. Safety on o asks the block's value to be constant on S_o alone, which is $L_o(x - y) = 0$ for all $x, y \in S_o$ — and over $\mathbb{F}_2$, $x - y$ is $x + y$. An empty S_o satisfies this with nothing to check: that is an unreachable record, whose map is zero and whose drop is safe. When a reachable S_o is a coset $x_0 + V$ of a linear subspace V, constancy on it is $L_o V = 0$, which the same method reads off from the baseline x_0 and the inputs $x_0 + v$, one run per basis vector v of V. Records defined only by equalities of affine bits have this coset form when reachable; a test naming

several levels need not, nor its complement, so the measurement-free polynomial bound does not by itself settle arbitrary lifted programs.

For the full classical gate set, an unsafe site on some record is witnessed by two basis inputs that reach that site with the same record and different block values. The witness is polynomially checkable. Deciding safety at every site without loans, on every record, is therefore still coNP-complete, with hardness already in the measurement-free fragment.

Liu, Zhou and Meng decide a neighbouring question about the same programs [29]. Their *safe* use of a clean ancilla, $U(|0\rangle_a|\varphi\rangle) = |0\rangle_a|\psi\rangle$ for every φ [29, Eq. (1)], is the criterion with the certificate pinned to the allocation ray; their *uncomputation* problem asks for a unitary that restores the ancilla and leaves the data in the sum of the branches.

Proposition 7.3 (the uncomputation condition, after [29, Def. 3.2, Prop. 3.1]). *Let G be a unitary on ancillas a and data q, write $G(|0\rangle_a|\varphi\rangle) = \sum_i |i\rangle_a|\varphi_i\rangle$, and let $F = \mathrm{func}(G, a)$ send $|\varphi\rangle$ to $\sum_i|\varphi_i\rangle$. Some unitary $\mathcal{U}$ has $\mathcal{U}(|0\rangle_a|\varphi\rangle) = |0\rangle_a F|\varphi\rangle$ for every φ if and only if F is unitary.*

Proof. F is linear. If it is unitary, $\mathcal{U} = I \otimes F$. Conversely $\|F\varphi\| = \||0\rangle_a \otimes F\varphi\| = \|\mathcal{U}(|0\rangle_a|\varphi\rangle)\| = \|\varphi\|$, so F is an isometry of a finite-dimensional space, hence unitary. The unitary G enters only through F. □

At a safe site whose rest consists exactly of the data, W is unitary, and every branch is $K_i = g_i W$. Their F is the sum of the branches, so here it is $(\sum_i g_i)W$, which with W unitary is itself unitary exactly when $|\sum_i g_i| = 1$. By proposition 7.3 their problem therefore has a solution exactly on that condition — a condition on the certificate that safety does not mention. Deutsch–Jozsa has $\sum_i g_i = 0$, its certificate being $|-\rangle$, whose two entries cancel. Program (a) is their own invalid example, whose branch sum "has norm $\sqrt{2}$" [29, §3.1], and is safe here. The copy $t := t \oplus v$ has branch sum I and is unsafe here. Neither predicate implies the other; their coNP-completeness [29, Cor. 3.1] prices the uncomputation problem, and theorem 7.1 prices safety.

7.3 The exact price

Planned What deciding the verdict costs over a fixed finite gate set, where safety is settled exactly from the circuit rather than estimated from runs. Being written; nothing here is claimed.

8 Implementation

The Haskell package has two decision engines. The modules under `src/Drop/Theory/` build the site map K and test proportionality of its Kraus operators, equivalently the Gram test of proposition 4.4. Its cost is exponential in the program's width, the number of qubits it holds at once, because the site map is built as a matrix on all of them. The modules under `src/Drop/Lang/` implement the language of section 3: parser, type checker, five-rule machine, site tests by state factorization on probe inputs, and compiler. The command `dqt compile programs/dj.s` emits OpenQASM 3.

`Drop.Bridge` compares the engines on the seventeen programs of the theory engine's one-temporary catalogue (`dqt list`) and the per-record table of section 5.9, with no disagreement. This catalogue is distinct from the nineteen source programs of table 1. The comparison shares exact scalar arithmetic and varies the decision algorithm. The engines use $\mathbb{Q}(\zeta_8)$ and the tower $\mathbb{Q}(\zeta_{2^k})$. A source scan excludes inexact numeric types from the decision code; the numerical gate emitter and its test are exempt.

program	site	d	n	lifts (register:block S)	verdict per record, certificate	reclaiming gate
`bell-block.s`*	t	4	2	—	safe, $(1,0,0,1)$	reflection (its `mat`)
`bell-pair.s`	(a,b)	4	2	—	safe, $(1,0,0,1)$	reflection
`bell-pair-split.s`	a,b	2	2	—	unsafe at both sites	—; —
`c.s`*	t	2	2	t:{0}	rec. 1: safe, $(1,0)$; rec. 0: unsafe	I; —
`copy-input.s`	x	2	2	—	unsafe (an input dropped)	—
`dj.s`	t	2	2	—	safe, $(1,-1)$	$(Z-X)/\sqrt{2}$
`grover.s`	t	2	4	—	safe, $(1,0)$	I
`learns-nothing.s`	t	2	2	—	unsafe	—
`mcx-dirty.s`	a	2	16	—	safe, loan handed back, $B = I$	I
`mcx-dirty-x.s`	a	2	16	—	safe, loan handed back by X	X
`mcx-dirty-half.s`	a	2	16	—	unsafe (4th Toffoli deleted)	—
`qpe.s`	ψ	2	8	—	safe, $(0,1)$	X
`rus.s`†	t	3	3	t:{0, 1}	rec. 1: unsafe; rec. 0: safe, $(0,0,1)$	—; $\|0\rangle \leftrightarrow \|2\rangle$
`shor.s`	w	16	8	w:{7}	rec. 1: safe, $\|7\rangle$; rec. 0: unsafe	$\|0\rangle \leftrightarrow \|7\rangle$; —
`simon.s`	w	4	4	w:{0}	rec. 1: safe, $(1,0,0,0)$; rec. 0: safe, $(0,1,0,0)$	I; $\|0\rangle \leftrightarrow \|1\rangle$
`superdense.s`	a	2	2	a:{0}, b:{0}	11,10: unreachable; 01,00: safe, $(0,1)$	—; X
`teleport.s`	m,a	2	2	m:{0}, a:{0}	at both sites every record safe, $(1,0)$ or $(0,1)$ by record	I or X
`temp-and.s`	a	2	4	a:{1}	rec. 1: safe, $(0,1)$; rec. 0: safe, $(1,0)$	X; I
`temp-general.s`	a	2	4	a:{1}	every record safe, $(1,0)$	I

Table 1. Every drop site of the nineteen programs (section 8.1); † outside the compiler's domain, * outside its gate table; d is the dropped block's dimension. The input dimension n includes all declared inputs except the block's loans; those loans are included in d. The last column is the reclaiming gate of propositions 4.11 and 4.12 per record: I means no gate, $|i\rangle \leftrightarrow |j\rangle$ a basis transposition, X is the qubit transposition, and a displayed matrix or "reflection" specifies the other gates. For loans these gates undo B. A dash marks an unsafe record (no certificate) or an unreachable one (every unit vector certifies it). The unparenthesized a,b and m,a entries each list two separate sites. A certificate is a vector or ket on its ray; a record lists the bits of the lifts before the site, 1 meaning the block of levels S survived.

8.1 The surface corpus

The corpus is a set of witnesses, not a performance benchmark. The three-level `rus.s` has dimension outside powers of two. The programs `temp-and.s` and `temp-general.s` both uncompute an AND by measurement and leave the visible input unchanged. Their corrections differ: one `diag` versus two extra Toffolis, with different final temporary basis states.

Silq's transcripts on the textbook programs: it refuses Deutsch–Jozsa and phase estimation, and Bernstein–Vazirani, the same kickback, with *cannot synthesize forget expression*; accepts Grover, superdense coding and teleportation; and on repeat-until-success type-checks only with a hand-supplied value witness, which fails with *bad forget* on six of twenty seeds.

8.2 What compiles, and what the compiler emits

Sixteen of the nineteen programs compile to OpenQASM 3; `rus.s` is refused by dimension, not approximated, and `bell-block.s` and `c.s` are refused for a gate outside the emitter's table, a 4×4 `mat` and the declared V. The simulation check also includes seven additional compiler cases. Across the 23 emitted programs it compares 173 leaves against the machine's reference output and finds no mismatch, up to global phase at tolerance 10^{-9}. The reference shares the reclaiming-factor code, so this checks emission rather than independently validating factor extraction. The Bell-pair case emits the five gates shown in fig. 4(b). The remaining examples use the cases of section 3.6: a single U on one qubit, or a basis transposition on a wider block. For basis strings at Hamming distance $d \geq 1$, the transposition uses $2d-1$ lines, and none at $d = 0$. The `simon.s` and `shor.s` examples use one and five lines respectively. Table 1 records the identity and loan cases; unsafe drops emit no reclaiming gate.

Table 2. Scope-end conditions. Our criterion permits a local reclaiming gate after the site (propositions 4.11 and 4.12).

system	scope end	the line it draws	what it asks
Selinger's flow charts [36]	`discard`	none: a partial trace	nothing; the register is never reused
Proto-Quipper [27]	`free`	none: a partial trace	nothing
Unruh et al. [40]	`Rel`	none: a partial trace	nothing
ReQWIRE [35]	`assert`	the allocation state	a semantic condition, then syntactic ones, sufficient not necessary
Silq [6]	automatic uncomputation	a recomputable value	a `qfree` producer
Qutes [11]	reclamation point	the allocation state	not live; no persistent irreversible effects
Twist [48]	purity type	purity	a semantic *iff* in Twist's core calculus, one-way with assertions
Guppy [24]	ownership, borrowing	ownership of values	that a value be unentangled from what it does not own
this paper	`drop`	fixed pure state; isometric loan factor	exact per record; a local reclaiming gate

9 Related work

Bona's body condition $F \otimes I$ is our loan condition with $B = I$ up to phase. We also permit any isometric block factor B, undone by a local unitary; a joint CZ between a loan and data fails both conditions.

For clean temporaries, call a program *restored* when it returns the block to its allocation state, and *restorable* when a continuation can do so. With unrestricted gates, every isometric site is restorable when $\dim[\![\Omega]\!] \le \dim \mathcal{H}_{\mathsf{rest}}$: extend the isometry that maps its image to the allocation state beside the embedded input. Requiring the continuation to act only on the block gives our criterion; restricting its coefficients adds the conditions of section 6.

Table 2 compares the scope constructs. Cavy and Qurts track ownership and borrowing [18, 30]; Qunity supports restrictions on error, space and purity [41].

Fact 9.1 (safe reclamation point [11, Def. 6]). *A program point p is a safe reclamation point for a variable v when v is not live at p and no irreversible operation applied to v has effects that persist beyond p. At such a point the compiler may insert the adjoint of the unitary history applied to v, restoring it to its initial state.*

Ancilla safety at circuit level. Li et al. reduce clean and dirty ancilla restoration to commutation with a reflection [28]. Liu et al. relate the difficulty of uncomputation to clean-ancilla safety [29, Prop. 3.3]; proposition 7.3 compares their functionality condition with ours. Su et al. give finite test states that establish the universal identity-on-the-loan condition [38]. Bona schedules substitutions among registers already certified safe [45].

Reuse by measurement. Bennett reuses a register by uncomputing it [4, 5]; DeCross et al. and CaQR by measuring it early and resetting it [10, 20].

Reading the outcome. Chen et al. decide, from the sound side only, when a measured outcome cannot change what a host program returns [8], the side corollary 5.7 forces on a record-oblivious

verdict; Yang and Yuan price checking assertions against a run [46], the cost shape of section 5.9's per-record table.

Mathematical foundations. Our matrix test applies standard Kraus and Choi rank theory to a drop site [9, 25, 43]. The Knill–Laflamme condition captures a weaker noninterference property (proposition B.1) [14, 23, 47]. Exact-synthesis column lemmas supply the ring-relative witness construction [1, 13].

Stabilizer representations also support entanglement analyses [2, 19, 34]. Twist also certifies Deutsch–Jozsa, using a programmer-supplied purity specification, static analysis and runtime checks [48, §9].

Language design. Typed registers and linear contexts follow QWIRE [33]. ReQWIRE supplies initialization and asserted release [35]. Fractional types provide the classical reuse-token account [7].

10 Conclusion

Safety characterizes when one unitary on the temporary block can restore its allocation state at every input. With measurement, the unitary is chosen separately for each record.

An experimental comparison with measurement and reset remains open. For a safe temporary block, both restore its allocation state and preserve the rest. Reset also permits reuse at unsafe temporary sites, but discards their correlations; it cannot in general return an unknown loan. Extending the criterion to recursion would also require an invariant that supplies one certificate across iterations.

References

[1] Matthew Amy, Andrew N. Glaudell, and Neil J. Ross. 2020. Number-Theoretic Characterizations of Some Restricted Clifford+T Circuits. *Quantum* 4, Article 252 (2020). doi:10.22331/q-2020-04-06-252

[2] Nicola Assolini, Alessandra Di Pierro, and Isabella Mastroeni. 2025. A Static Analysis of Entanglement. In *Verification, Model Checking, and Abstract Interpretation (VMCAI 2025) (LNCS, Vol. 15530)*. 50–71. doi:10.1007/978-3-031-82703-7_3

[3] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter. 1995. Elementary Gates for Quantum Computation. *Physical Review A* 52, 5 (1995), 3457–3467. doi:10.1103/PhysRevA.52.3457

[4] Charles H. Bennett. 1973. Logical Reversibility of Computation. *IBM Journal of Research and Development* 17, 6 (1973), 525–532. doi:10.1147/rd.176.0525

[5] Charles H. Bennett. 1989. Time/Space Trade-Offs for Reversible Computation. *SIAM J. Comput.* 18, 4 (1989), 766–776. doi:10.1137/0218053

[6] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. 286–300. doi:10.1145/3385412.3386007

[7] Chao-Hong Chen, Vikraman Choudhury, Jacques Carette, and Amr Sabry. 2020. Fractional Types: Expressive and Safe Space Management for Ancilla Bits. In *Reversible Computation — 12th International Conference, RC 2020 (Lecture Notes in Computer Science, Vol. 12227)*. Springer, 169–186. doi:10.1007/978-3-030-52482-1_10

[8] Yanbin Chen, Qunyou Liu, Yu Wang, Christian B. Mendl, and Helmut Seidl. 2026. GPU-Accelerated Host-Aware Dead-Measurement Detection in Hybrid Quantum–Classical Programs. arXiv:2607.09976 [quant-ph] To appear at SAS 2026; full version.

[9] Man-Duen Choi. 1975. Completely Positive Linear Maps on Complex Matrices. *Linear Algebra Appl.* 10, 3 (1975), 285–290. doi:10.1016/0024-3795(75)90075-0

[10] Matthew DeCross, Eli Chertkov, Megan Kohagen, and Michael Foss-Feig. 2023. Qubit-Reuse Compilation with Mid-Circuit Measurement and Reset. *Physical Review X* 13, 4 (2023), 041057. doi:10.1103/PhysRevX.13.041057

[11] Simone Faro, Francesco Pio Marino, and Gabriele Messina. 2026. Reversible Lifetime Semantics for Quantum Programs. arXiv:2603.14538 [cs.PL]

[12] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2023. Proto-Quipper with Dynamic Lifting. *Proceedings of the ACM on Programming Languages* 7, POPL (2023). doi:10.1145/3571204

[13] Brett Giles and Peter Selinger. 2013. Exact Synthesis of Multiqubit Clifford+T Circuits. *Physical Review A* 87, 3 (2013), 032332. doi:10.1103/PhysRevA.87.032332 arXiv:1212.0506; Lemma 5 is the column lemma.

[14] Joseph A. Goguen and José Meseguer. 1982. Security Policies and Security Models. In *1982 IEEE Symposium on Security and Privacy*. doi:10.1109/SP.1982.10014

[15] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. 2013. Quipper: A Scalable Quantum Programming Language. *ACM SIGPLAN Notices (PLDI)* 48, 6 (2013), 333–342. doi:10.1145/2499370.2462177

[16] Robert Harper. 2016. *Practical Foundations for Programming Languages* (second ed.). Cambridge University Press.

[17] Chris Heunen and Robin Kaarsgaard. 2022. Quantum Information Effects. *Proc. ACM Program. Lang.* 6, POPL, Article 2 (2022). doi:10.1145/3498663

[18] Kengo Hirata and Chris Heunen. 2025. Qurts: Automatic Quantum Uncomputation by Affine Types with Lifetime. *Proceedings of the ACM on Programming Languages* 9, POPL, Article 6 (2025), 155–182 pages. doi:10.1145/3704842

[19] Kentaro Honda. 2015. Analysis of Quantum Entanglement in Quantum Programs using Stabilizer Formalism. In *Proceedings of the 12th International Workshop on Quantum Physics and Logic (QPL 2015) (EPTCS, Vol. 195)*. 262–272. doi:10.4204/EPTCS.195.19

[20] Fei Hua, Yuwei Jin, Yanhao Chen, Suhas Vittal, Kevin Krsulich, Lev S. Bishop, John Lapeyre, Ali Javadi-Abhari, and Eddy Z. Zhang. 2023. CaQR: A Compiler-Assisted Approach for Qubit Reuse through Dynamic Circuit. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS 2023)*. 59–71. doi:10.1145/3582016.3582030

[21] Stephen P. Jordan. 2014. Strong Equivalence of Reversible Circuits Is coNP-Complete. *Quantum Information and Computation* 14, 15–16 (2014), 1302–1307. arXiv:1307.0836

[22] Vadym Kliuchnikov, Dmitri Maslov, and Michele Mosca. 2013. Fast and Efficient Exact Synthesis of Single-Qubit Unitaries Generated by Clifford and T Gates. *Quantum Information and Computation* 13, 7–8 (2013), 607–630. doi:10.26421/QIC13.7-8-4

[23] Emanuel Knill and Raymond Laflamme. 1997. Theory of Quantum Error-Correcting Codes. *Physical Review A* 55, 2 (1997), 900–911. doi:10.1103/PhysRevA.55.900

[24] Mark Koch, Agustín Borgna, Craig Roy, Alan Lawrence, Kartik Singhal, Seyon Sivarajah, and Ross Duncan. 2025. Imperative Quantum Programming with Ownership and Borrowing in Guppy. arXiv:2510.13082 [cs.PL] Presented at PLanQC 2025.

[25] Karl Kraus. 1983. *States, Effects, and Operations: Fundamental Notions of Quantum Theory*. Lecture Notes in Physics, Vol. 190. Springer, Berlin, Heidelberg. doi:10.1007/3-540-12732-1

[26] Dennis Kretschmann, Dirk Schlingemann, and Reinhard F. Werner. 2008. The Information-Disturbance Tradeoff and the Continuity of Stinespring's Representation. *IEEE Transactions on Information Theory* 54, 4 (2008), 1708–1717. doi:10.1109/TIT.2008.917696

[27] Dongho Lee, Valentin Perrelle, Benoît Valiron, and Zhaowei Xu. 2021. Concrete Categorical Model of a Quantum Circuit Description Language with Measurement. In *41st IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2021) (LIPIcs)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.FSTTCS.2021.13

[28] Jiqi Li, Jingyi Mei, Wang Fang, and Ji Guan. 2026. Formal Verification of Quantum Ancilla Safety. In *Computer Aided Verification*. Springer Nature Switzerland, 326–348. doi:10.1007/978-3-032-32537-2_16 arXiv:2608.13099.

[29] Chenke Liu, Li Zhou, and Boning Meng. 2026. Quantum Uncomputation of Clean and Dirty Ancilla Qubits. arXiv:2608.09578

[30] Christopher Michael McNally. 2021. *Practical Modern Quantum Programming*. S.M. thesis. Massachusetts Institute of Technology. https://inspirehep.net/files/aa1f237c1d580cc362cf2bf89f5c6701

[31] Microsoft. 2025. Quantum Memory Management in Q#. Microsoft Learn, Azure Quantum documentation. https://learn.microsoft.com/en-us/azure/quantum/user-guide/language/statements/quantummemorymanagement

[32] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: Synthesizing Uncomputation in Quantum Circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*. ACM, 222–236. doi:10.1145/3453483.3454040

[33] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A Core Language for Quantum Circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL '17)*. 846–858. doi:10.1145/3009837.3009894

[34] Simon Perdrix. 2008. Quantum Entanglement Analysis Based on Abstract Interpretation. In *Static Analysis, 15th International Symposium (SAS 2008) (LNCS, Vol. 5079)*. 270–282. doi:10.1007/978-3-540-69166-2_18

[35] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. *Electronic Proceedings in Theoretical Computer Science* 287 (2019), 299–312. doi:10.4204/EPTCS.287.17

[36] Peter Selinger. 2004. Towards a Quantum Programming Language. *Mathematical Structures in Computer Science* 14, 4 (2004), 527–586. doi:10.1017/S0960129504004256

[37] Peter Selinger and Benoît Valiron. 2006. A Lambda Calculus for Quantum Computation with Classical Control. *Mathematical Structures in Computer Science* 16, 3 (2006), 527–552. doi:10.1017/S0960129506005238

[38] Bonan Su, Li Zhou, Yuan Feng, and Mingsheng Ying. 2025. Borrowing Dirty Qubits in Quantum Programs. arXiv:2508.17190 [quant-ph]

[39] Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. 2018. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop 2018 (RWDSL2018)*. Article 7, 10 pages. doi:10.1145/3183895.3183901

[40] Dominique Unruh, Benoît Valiron, and Mingsheng Ying. 2026. Reasoning about Parameterized Recursive Quantum Programs with Ancilla Data and Probabilistic Control. *ACM Transactions on Computational Logic* 27, 4, Article 23 (2026), 63 pages. doi:10.1145/3815573

[41] Finn Voichick, Liyi Li, Robert Rand, and Michael Hicks. 2023. Qunity: A Unified Language for Quantum and Classical Computing. *Proceedings of the ACM on Programming Languages* 7, POPL (2023), 921–951. doi:10.1145/3571225

[42] Lawrence C. Washington. 1997. *Introduction to Cyclotomic Fields* (2 ed.). Graduate Texts in Mathematics, Vol. 83. Springer. doi:10.1007/978-1-4612-1934-7

[43] John Watrous. 2018. *The Theory of Quantum Information*. Cambridge University Press. doi:10.1017/9781316848142 Theorem 2.22 and Corollaries 2.23, 2.27 for the operator-sum form and its freedom; the Choi rank is defined on p. 78 and isometric channels on p. 91.

[44] Howard M. Wiseman and Gerard J. Milburn. 2010. *Quantum Measurement and Control*. Cambridge University Press. doi:10.1017/CBO9780511813948

[45] Xiaoquan Xu, Chenke Liu, Boning Meng, Zihao Shen, and Li Zhou. 2026. Bona: Automatic Management of Dirty Ancilla Borrowing in Quantum Circuits. arXiv:2608.08765 [cs.PL]

[46] Shengyuan Yang and Charles Yuan. 2026. The Time-Space Complexity of Checking Multiple Assertions in Quantum Programs. arXiv:2607.11665 [cs.PL]

[47] Mingsheng Ying, Yuan Feng, and Nengkun Yu. 2013. Quantum Information-Flow Security: Noninterference and Access Control. In *2013 IEEE 26th Computer Security Foundations Symposium*. 130–144. doi:10.1109/CSF.2013.16

[48] Charles Yuan, Christopher McNally, and Michael Carbin. 2022. Twist: Sound Reasoning for Purity and Entanglement in Quantum Programs. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 30 (2022), 32 pages. doi:10.1145/3498691

A Proofs for the language section

Proof of theorem 3.3. *Progress.* A term that is not `return` v begins with a statement, and each of the three heads has exactly one rule; typing puts the registers v names in the context, so U_v is defined. *Preservation*, per rule: at Step-Gate $U_v\sigma \in [\![\Delta]\!]$ and the continuation types in the same context, since the gate returned the registers under their own names; at Step-New $\sigma \otimes \psi_w \in [\![\Delta{\cdot}w]\!]$ and the continuation types in $\Omega, w : [d]$ by remark 3.2, using the rule's binder-freshness condition; at Step-Drop σ is unchanged and the continuation types in the context the statement left. *Termination*: each rule removes one statement, so the run takes exactly $|M|$ steps. *Adequacy*: by induction on the run, the state after k steps is the composite of the first k per-step maps applied to the input. At termination that composite is $[\![M]\!]$. The allocated registers are kept in allocation order. Each operation acts on its named factors, in the tuple order specified by the term; the denotation uses the same convention. □

Proof of lemma 3.4. Induction on M_1. At `return` v, clause (1) is remark 3.2 applied once per name of $\bar{y}$ — which is where the disjointness of $\Omega_{\bar{y}}$ from Ω_1 is used, since without it the second renaming would rename a name the first produced — and clause (2) is the definition of $+\!\!+$. At a statement, the statement is unchanged and the induction hypothesis handles the tail; the freshness conditions are what let the `new` clause push its binder past M_2 and the `lift` clause substitute its bit without reaching into M_2. At an `if`, both arms are sequenced and the guard is untouched. Clause (3): sequencing appends M_2 to every branch of M_1, so the longest branch grows by $|M_2|$. □

Proof of theorem 3.5. Induct over the source statements. Allocation prepares its declared state, and each source gate emits the same matrix up to its row phase. An unsafe drop emits nothing, as in Step-Drop. A safe drop emits its reclaiming gate up to a phase. That gate acts only on the dropped registers. Typing prevents every later statement from naming them, so it commutes with

the tail of the circuit. Moving each reclaiming gate to the end yields the displayed tensor product, multiplied by the product of all row phases. The fresh-allocation premise ensures these blocks are disjoint.

For clause (2), trace out the dropped registers. Unitaries on those registers leave this partial trace unchanged, and the global phase cancels in the density matrix. The returned state is therefore unchanged.

For clause (3), write a temporary site's map as $\tilde{g} \otimes W$. Its reflection sends $\tilde{g}$ to ψ, so an emitted phase c gives $\psi \otimes (cW)$. The block's state is exactly ψ. At a loan-only site, $B^\dagger B = I$ returns every loan input unchanged, including its correlations with an external register. At an emittable mixed site, $B = B' \otimes g$, the adjoint on the loans and canonical reflection on the temporaries give the same conclusion. The split phase is absorbed into the rest factor. None of these state statements requires the circuit's global phase to be one. □

B Proofs for the criterion section

Unless a statement allows a general linear map, s is a measurement-free site as in theorem 4.10: the block is allocated, so $\mathcal{H}_{\text{rest}} = [\![\Omega]\!] \otimes \mathcal{H}_{\text{other}}$ with $\mathcal{H}_{\text{other}}$ the other allocated registers, $K = \sigma_s : [\![\Omega]\!] \to \mathcal{H}_v \otimes \mathcal{H}_{\text{rest}}$ is an isometry, $K_i = (\langle i| \otimes I)K$, $n = \dim[\![\Omega]\!]$, $m = \dim \mathcal{H}_{\text{rest}}$, $d = \dim \mathcal{H}_v$, and $\hat{K}$ is the $d \times mn$ matrix whose i-th row is K_i written out, so that $\Gamma^\top = \hat{K}\hat{K}^\dagger - \hat{K}$ is that stacked matrix throughout, and $\hat{K}_i$ its i-th row. The block follows the drop-tuple order and the rest follows allocation order, as in definition 4.1. Named-factor operations include the corresponding reordering when written as matrices.

Proof of proposition 4.4. (1) $K = \sum_i |i\rangle \otimes K_i$, so $K = g \otimes W$ exactly when $K_i = g_i W$ for all i, that is when the K_i span a line or are all zero; this is rank $\hat{K} \le 1$, and rank $\hat{K} = \operatorname{rank} \hat{K}\hat{K}^\dagger = \operatorname{rank} \Gamma$. Now $\Gamma \succeq 0$, and a positive semidefinite matrix has rank at most one exactly when it is $\operatorname{tr}\Gamma$ times a rank-one projector or zero, that is when $\Gamma^2 = (\operatorname{tr}\Gamma)\,\Gamma$: its eigenvalues then lie in $\{0, \operatorname{tr}\Gamma\}$ and sum to $\operatorname{tr}\Gamma$. At the site $\operatorname{tr}\Gamma = \operatorname{tr} K^\dagger K = n$, K being an isometry. For $K \neq 0$, the input I/n gives the unnormalized joint state $KK^\dagger/n$, of trace $\operatorname{tr}\Gamma/n$, so normalised $KK^\dagger/\operatorname{tr}\Gamma$, whose partial trace over the rest has (i,j) entry $\operatorname{tr}(K_i K_j^\dagger)/\operatorname{tr}\Gamma = \Gamma_{ji}/\operatorname{tr}\Gamma$: the block's state is $\Gamma^\top/\operatorname{tr}\Gamma$, pure exactly when its rank is one, and $gg^\dagger$ when $K = g \otimes W$.

(2) Write $\Phi(\rho) = \sum_i K_i \rho K_i^\dagger$ for the completely positive map the drop leaves on the rest, and S for the span of the operators $K_0, \dots, K_{d-1}$ inside the space of linear maps $[\![\Omega]\!] \to \mathcal{H}_{\text{rest}}$, an inner-product space under $\langle A, B\rangle = \operatorname{tr}(A^\dagger B)$. The channel determines S. For any operator A, the number $\sum_{b,a} \langle Ae_b,\ \Phi(|e_b\rangle\langle e_a|)\, Ae_a\rangle$ is computed from Φ alone, and expanding Φ along any family (L_k) representing it gives $\sum_k |\langle A, L_k\rangle|^2$. Thus every representing family has the same orthogonal complement, hence the same span S.

$\dim S$ is the rank of Γ: writing each K_i out as a vector makes Γ their Gram matrix, whose rank is the dimension of their span. Any representing family spans S, so it has at least $\operatorname{rank}\Gamma$ operators. A block needs at least that many levels.

For $K \neq 0$, that many levels suffice. Let $c \mapsto \sum_i c_i K_i$ be the combination map $\mathbb{C}^d \to S$, let $y_1, \dots, y_r$ be an orthonormal basis of the orthogonal complement of its kernel, $r = \dim S$, and put $L_k = \sum_i (y_k)_i K_i$. For each j the vector $\sum_k \overline{(y_k)_j}\, y_k$ agrees with e_j modulo that kernel, so the L_k have the channel Φ. Stacking them, $Vx = \sum_k |k\rangle \otimes L_k x$ is a site map on an r-level block with channel Φ, and $\|Vx\|^2 = \operatorname{tr}\Phi(xx^\dagger) = \|Kx\|^2$, so V is an isometry exactly when K is. Appending zero operators to the family widens the block without changing the channel or any norm, so every positive $r \ge \operatorname{rank}\Gamma$ is realized. Finally, when K is an isometry, as at a site of a measurement-free

prefix, so is V, and lemma 4.6 extends $|0\rangle \otimes x \otimes |0 \cdots 0\rangle \mapsto Vx$ to a unitary of $\mathbb{C}^r \otimes [\![\Omega]\!] \otimes \mathcal{H}_{\text{other}}$: allocating the block and the other registers in $|0\rangle$, applying it and dropping the block is a program with site map V. For any linear K the count stands, of the site maps that realise the channel. For $K = 0$, use one zero Kraus operator on the no-register space $\mathbb{C}$. The empty Kraus family represents the same zero channel.

(3) A gate C on the rest replaces K_i by CK_i and leaves $\operatorname{tr}(K_i^\dagger C^\dagger C K_j) = \Gamma_{ij}$; a gate U on the block replaces K_k by $K'_k = \sum_i U_{ki} K_i$, and $\operatorname{tr}(K'^\dagger_k K'_l) = \sum_{ij} \bar{U}_{ki} U_{lj} \Gamma_{ij} = (\bar{U} \Gamma U^\top)_{kl}$. □

Proposition B.1 (the block may learn nothing and the drop be unsafe). *Let $K = \sigma_s$ be the isometric map of a measurement-free site with no loans in its block. Use the operators K_i of proposition 4.4. The block has the same reduced state at every unit input exactly when*

$$K_i^\dagger K_j = c_{ij} I \qquad \text{for all } i, j.$$

Under this condition the drop is safe exactly when c has rank one. Then $c_{ij} = \bar{g}_i g_j$ for a unit certificate g. In contrast, the program below has the constant mixed state $c = I/2$ and is unsafe.

The first sentence is the Knill–Laflamme condition [23] with $[\![\Omega]\!]$ as the code and the block as the environment, which therefore learns nothing about the input [26]: noninterference from the input to the block [14, 47]. It says the block's state is the same at every input; safety says that this one state is pure, and it need not be once the rest can hold a register entangled with the block. With $x : [2]$ the input, allocate a and t in $|0\rangle$, apply H to t and CX from t to a, and drop t: the block's state is $I/2$ at every input, $K_0^\dagger K_1 = 0$ and $K_i^\dagger K_i = I/2$, and the drop is unsafe, t being entangled with a. What a drop must not disturb is everything else in the machine, not the input alone.

Proof of proposition B.1. If $K_i = g_i W$ with W an isometry then $K_i^\dagger K_j = \bar{g}_i g_j W^\dagger W = \bar{g}_i g_j I$. The block's state at a unit input x has entries $\langle K_j x, K_i x \rangle = x^\dagger K_j^\dagger K_i x$, and over $\mathbb{C}$ a matrix whose form $x \mapsto x^\dagger A x$ is constant on unit vectors is a scalar, by polarisation; so the state is input-independent exactly when every $K_j^\dagger K_i$ is a scalar.

For the converse let $K_i^\dagger K_j = c_{ij} I$. Each coefficient is an overlap of two of the K_i at one basis input, $c_{ij} = \langle K_i e_a, K_j e_a \rangle$, so c is hermitian and positive semidefinite, and $c^\top$ is the block's state at every unit input; $\operatorname{tr} c = 1$ because $\sum_i K_i^\dagger K_i = K^\dagger K = I$. If the drop is safe with certificate g then $c_{ij} = \bar{g}_i g_j$, of rank one. Conversely let c have rank one. A hermitian positive semidefinite matrix of rank one is an outer product, $c_{ij} = \bar{g}_i g_j$ with g a unit vector because $\operatorname{tr} c = 1$, and the Gram matrix is $\Gamma = n\,c$, with entries $\Gamma_{ij} = n \bar{g}_i g_j$. Vectors whose Gram matrix is an outer product are proportional with its weights: the vector $\hat{K}_i - (g_i / g_{i_0}) \hat{K}_{i_0}$ has norm zero, so $\hat{K}_i = \sqrt{n}\, g_i V$ for one V, and $K_i = g_i W$ for the operator W that V writes out — which is safety with certificate g. Nothing but $\operatorname{tr} c = 1$ is used of the isometry, and it is used only to exclude $c = 0$: with no hypothesis on the site map the criterion reads *rank at most one*, and the zero site map of an unreachable record has $c = 0$, is safe, and is certified by every unit vector of the block. For the Bell example, $c = I/2$ has rank two; proposition 4.4(2) also shows that its channel needs at least two block levels. □

Proposition B.2 (a product at every input). *Let $K : [\![\Omega]\!] \to \mathcal{H}_\nu \otimes \mathcal{H}_{\text{rest}}$ be linear, with $n = \dim [\![\Omega]\!]$, and suppose Kx is a product vector, or zero, for every x. Then K has one of two shapes, and both exactly when K has rank at most one. Here the language spaces $\mathcal{H}_\nu$ and $\mathcal{H}_{\text{rest}}$ have positive dimension; the empty register tuple denotes $\mathbb{C}$.*

the temporary is fixed $K = g \otimes W$ *for a vector* $g \in \mathcal{H}_v$ *and a linear* $W : [\![\Omega]\!] \to \mathcal{H}_{\text{rest}}$ *— the rest varies, the temporary does not;*

the rest is fixed $Kx = Gx \otimes w$ *for a fixed nonzero* $w \in \mathcal{H}_{\text{rest}}$ *and a linear* $G : [\![\Omega]\!] \to \mathcal{H}_v$ *— the temporary varies, the rest does not.*

The first shape is what definition 4.1 asks of a site map. A map of the second shape is $g \otimes W$ *exactly when* G *has rank at most one, so such an isometry with* $n \geq 2$ *never is.*

Proof of proposition B.2. For each x with $Kx \neq 0$ write $Kx = g_x \otimes w_x$ with g_x, w_x nonzero. Either all the g_x are parallel or two of them are not.

If every g_x lies on the line of one unit vector g, then $Kx \in g \otimes \mathcal{H}_{\text{rest}}$ for every x and lemma 4.3(1) gives $K = g \otimes W$.

Otherwise choose x_1, x_2 with g_{x_1}, g_{x_2} linearly independent. By linearity $K(x_1 + x_2) = g_{x_1} \otimes w_{x_1} + g_{x_2} \otimes w_{x_2}$, which — writing a vector of $\mathcal{H}_v \otimes \mathcal{H}_{\text{rest}}$ as the $d \times m$ matrix of its coefficients — is $g_{x_1} w_{x_1}^\top + g_{x_2} w_{x_2}^\top$, of rank two unless w_{x_1} and w_{x_2} are parallel; a product vector, or zero, has rank at most one, so they are parallel to one w. Any x_3 with $Kx_3 \neq 0$ has g_{x_3} independent of g_{x_1} or of g_{x_2}, and the same argument makes w_{x_3} parallel to w. So $G = (I \otimes \langle w|)K/\|w\|^2$ is linear with $Kx = Gx \otimes w$ at every x with $Kx \neq 0$, and trivially at $Kx = 0$.

For the last sentence, $K_i x = (Gx)_i\, w$, so $K_i = w\, e_i^\top G$, and these are pairwise proportional exactly when the rows of G are, that is, when G has rank at most one. If K is an isometry then $\|Gx\|\,\|w\| = \|x\|$, so G is injective, of rank n. □

Proof of lemma 4.3. (1) $\sigma(x) = g \otimes w(x)$ for a unique $w(x)$, and $w = (\langle g| \otimes I)\sigma$ is linear and has $\|w(x)\| = \|g \otimes w(x)\| = \|\sigma(x)\|$. (2) Every x with $Bx = 0$ has $Ax = 0$, so both maps descend to $X/\ker B$, where the descent of B is injective and a scalar proved there is the scalar here; so assume B injective. For $x \neq 0$ write $Ax = c(x)Bx$, the scalar unique because $Bx \neq 0$. If $y = \alpha x$ with $\alpha \neq 0$ then $Ay = \alpha Ax = c(x)By$, so $c(y) = c(x)$. If x and y are independent then so are Bx and By, and $c(x + y)(Bx + By) = c(x)Bx + c(y)By$ forces $c(x) = c(x + y) = c(y)$. So c is constant on $X \setminus \{0\}$. □

Proof of lemma 4.6. $V(S)$ has the dimension of S, so $S^\perp$ and $V(S)^\perp$ have the same dimension; let $V' : S^\perp \to V(S)^\perp$ be an isometry between them, given by a choice of orthonormal bases. Then $V \oplus V'$ carries an orthonormal basis of $\mathcal{H}$ to an orthonormal basis, so it is unitary, and restricts to V on S. □

Proof of proposition 4.2. Write the inputs as $\mathcal{H}_{\text{loan}} \otimes [\![\Omega']\!] \otimes \mathcal{H}_c$. If $\sigma_s = B \otimes W$ then $\sigma_s \otimes \mathrm{id} = B \otimes (W \otimes \mathrm{id})$, by linearity. Conversely, if $\sigma_s \otimes \mathrm{id} = B \otimes W'$, apply both sides to $u \otimes y \otimes |r_0\rangle$ for one basis state $|r_0\rangle$ of $\mathcal{H}_c$ and read the slice of the output at $|r_0\rangle$: $\sigma_s(u \otimes y) = Bu \otimes W(y)$ with W that slice. □

Proof of theorem 4.10.

⟨1⟩ (3) ⇒ (2) ⇒ (1) are the definitions: a witness sends x to $R\psi \otimes W(x \otimes |0 \cdots 0\rangle)$, the shape of (2) with $g = R\psi$, and (2) is the shape definition 4.1 asks for. (1) ⇒ (2): the W of definition 4.1 is $(\langle g| \otimes I)\,\sigma_s$ and is an isometry because $K = \sigma_s$ is (lemma 4.3(1)), the gates of definition 3.1 being unitary and `new` adjoining a unit vector.

⟨2⟩ (2) ⇒ (3). Lemma 4.6 extends the isometry $x \otimes |0 \cdots 0\rangle \mapsto Wx$ of the subspace $[\![\Omega]\!] \otimes |0 \cdots 0\rangle$ to a unitary of $\mathcal{H}_{\text{rest}}$. For the block, the same lemma extends $\psi \mapsto g$ to a unitary R. Then $\mathrm{sep}(R, W)$ is typed and denotes $g \otimes W = \sigma_s$.

⟨3⟩ (3) ⇒ (4): $[\![\mathrm{sep}^\circ(R,W)]\!] = \sigma_s = [\![M|_s]\!]$, and equal denotations give equal answers in every context. (4) ⇒ (2): lemma 4.9 gives $\sigma_s = c\,[\![\mathrm{sep}^\circ(R,W)]\!] = c\,(g \otimes W) = g \otimes cW$. The last sentence: $\sigma' = c\,\sigma$ by lemma 4.9, and proportional maps factor together, with the same g. □

Proposition B.3 (the composite temporary). *Let M have temporaries $t_1, \ldots, t_n$, each dropped once on the chosen record, and let Φ be its final state map on that record. For clause (1), assume that the tail after the drop of t_k is injective on the span of the row vectors $(\langle e_i| \otimes I)\sigma_s(x)$, as i and x vary. This holds if that tail contains no lifts. For a lift, the condition says that its projection kills no nonzero vector in the relevant span. Call t_k* separately constant *when $\Phi(x)$ lies in $g_k \otimes$ (the other factors) for one unit $g_k \in \mathcal{H}_{t_k}$ and every input x.*

(1) The site of drop t_k *is safe if and only if t_k is separately constant.*

(2) Every t_k is separately constant if and only if the composite site drop $(t_1, \ldots, t_n)$*, placed at the end after deleting the individual drops, is safe with a certificate carrying the product $g_1 \otimes \cdots \otimes g_n$.*

Proof of proposition B.3. After the site of drop t_k every step is $I_{t_k} \otimes T$ for a linear T on the other factors: a gate on live registers, none of which is t_k; an allocation, which adjoins a product factor; or a drop, which applies nothing; or a lift, a projection, whose kernel is the vectors supported off its block, so it is injective on the rows exactly when no nonzero combination of them vanishes there. So $\Phi = (I_{t_k} \otimes T) \circ \sigma_s$, with T injective on the rows of σ_s by hypothesis, as it is everywhere when no lift follows, being a composite of isometries. Without injectivity the converse can fail: lifting r after preparing a Bell pair on t_k, r makes the selected state a product. If $\sigma_s(x) \in g \otimes \mathcal{H}_{\mathrm{rest}}$ for every x then $\Phi(x) \in g \otimes$ (the rest). Conversely write $\sigma_s(x) = \sum_i e_i \otimes y_i(x)$ in a basis of $\mathcal{H}_{t_k}$, so $\Phi(x) = \sum_i e_i \otimes T y_i(x)$; membership in $g \otimes$ (the rest) says $T y_i(x) = g_i z$ for one z. Pick i_0 with $g_{i_0} \neq 0$ and put $y = y_{i_0}(x)/g_{i_0}$; then $T(y_i(x) - g_i y) = 0$ with $y_i(x) - g_i y$ in the span of the rows, so $y_i(x) = g_i y$, which is safety. That is (1). For (2), a product certificate makes every t_k separately constant at once; in the other direction $\Phi(x)$ lies in every $g_k \otimes$ (the others), and the intersection of those subspaces is $g_1 \otimes \cdots \otimes g_n \otimes [\![A]\!]$, as expanding in product bases containing each g_k shows. □

Proof of lemma 4.9. Write $\rho(x) = |\sigma x\rangle\langle\sigma x|$ and $\rho'(x) = |\sigma' x\rangle\langle\sigma' x|$ at unit x; both prefixes return every register, so neither has a hidden factor. Every unit x is prepared by a closed context — allocate the registers of Ω in $|0 \cdots 0\rangle$ and apply one unitary carrying $|0 \cdots 0\rangle$ to x, a gate because $\mathcal{G}$ is every unitary — and every unitary V on the output register is a context gate. Equal answers therefore give $\langle u|\rho(x)|u\rangle = \langle u|\rho'(x)|u\rangle$ at every unit $u = V^\dagger e_b$, and an operator with every such expectation zero is zero. Hence $\sigma' x \in \mathbb{C}\,\sigma x$ for every x, and lemma 4.3(2) gives $\sigma' = c\,\sigma$; equal answers also give $\|\sigma' x\| = \|\sigma x\|$, so $|c| = 1$ once $\sigma \neq 0$. Conversely a context is linear in its hole, so $\mathcal{C}\{M'\}$ denotes c times what $\mathcal{C}\{M\}$ does, and a weight, a squared norm, does not see $|c| = 1$. □

Lemma 4.9 uses exactly two properties of $\mathcal{G}$: that its contexts prepare a set of inputs dense in the unit sphere of $[\![\Omega]\!]$, after which continuity of $x \mapsto \rho(x)$ finishes; and *informational completeness*, that the operators $V^\dagger|e_b\rangle\langle e_b|V$ span the Hermitian operators on the output register. Clifford measurements span the Hermitian operators on any finite number of qubits: every Pauli string is diagonalized by local Clifford gates, and Pauli strings form an operator basis. Complex Householder reflections also give an informationally complete set; the *real* reflections do not, which is why the field is $\mathbb{C}$ throughout. Restricting output contexts to the identity loses this information: Z and I have the same basis-measurement probabilities at every input, but their maps are not globally proportional.

Proof of proposition 4.11. The reflection in the hyperplane orthogonal to $w = \psi - \tilde{g}$ exchanges ψ and $\tilde{g}$, because ψ and $\tilde{g}$ are unit vectors whose inner product $\langle\psi, \tilde{g}\rangle$ is real and lies in $[0, 1]$ by the

choice of $\tilde{g}$; so R^{ψ} is a unitary involution with $R^{\psi}\psi = \tilde{g}$ and $R^{\psi}\tilde{g} = \psi$. A linear map exchanging ψ and $\tilde{g}$ and fixing their orthogonal complement is determined everywhere, and R^{ψ} is one, w lying in their span. $R^{\psi} = I$ exactly when $w = 0$, that is when $\tilde{g} = \psi$, which by the choice of $\tilde{g}$ is exactly when g is a multiple of ψ: a multiple $\lambda\psi$ with $|\lambda| = 1$ has representative ψ. The site being safe, its state is $\tilde{g} \otimes Wx$ by lemma 4.3(1) once W absorbs the phase carrying g to $\tilde{g}$, and $(R^{\psi} \otimes I)(\tilde{g} \otimes Wx) = \psi \otimes Wx$ at every x. □

Proof of proposition 4.12. At a loan-only site, B is unitary, and $B^{\dagger}B\ell = \ell$ returns every loan vector unchanged. For a mixed block, group loans before temporaries and let $J\ell = \ell \otimes \psi$ embed the loan with the allocated temporaries. The same argument applies after permuting back to the drop-tuple order. The isometry $B\ell \mapsto J\ell$ between their images extends by lemma 4.6 to a unitary T on the block. Then $TB = J$. These equalities extend linearly to inputs entangled with the rest or an external reference. □

Proof of proposition 4.13. Compare the unnormalized reduced states on the live registers in the pooled and standard machines, also retaining any external reference. Trace out registers already hidden by the standard machine and registers still waiting in the pool. In addition, require each pooled register to be in its declared allocation state and uncorrelated with the live registers. These conditions hold initially.

At a safe temporary site, the block is in a fixed pure state g and separate from the rest. Reclaiming it puts it in ψ, ready for the pool; hiding it in the standard machine leaves the same reduced state on the remaining live registers. Serving this pooled register to a later allocation therefore agrees with adjoining a fresh register in ψ. All other steps apply the same operation to the live registers of both machines. This includes each selected measurement projection, so the reduced states remain equal without normalization. Their traces give the same record weights, and their final reductions give the same returned states. A returned loan is preserved, together with its external correlations, by proposition 4.12. This proves (1); counting fresh allocations proves (2).

For (3), the state of program (b) at input $|0\rangle_v$ is not a product across the block (section 2), a gate on the block alone leaves it so, and a later allocation served from that block starts correlated with v where the standard machine adjoins a fresh $|0\rangle$ in a product with it; the program that returns both registers returns different states. □

C Proofs for the measurement section

States of program (c) and of proposition 5.8's pair are written with the temporary first.

Proof of theorem 5.1. Only the two rules of section 5.2 are new work, and every clause of theorem 3.3 carries over with them. A lift or an `if` at the head is covered by exactly one rule (both labelled instances at a lift), and removes one statement or replaces an `if` by one of its arms, so the number of statements along the branch the run follows drops by one at every step, which is clause (2). It keeps the state in $[\![\Delta]\!]$ and the named registers unchanged, and contributes its own map $-\,(P^{b}_{S})_{x}$, the identity — to the composite that adequacy reads along the record. Two cases carry the content. *At a lift*, the premise types M in Π, p; $\Omega, x : [d]$, the lift binding no register, and substituting the closed bit b for p types it in Π; $\Omega, x : [d]$, by induction on the typing derivation: the one rule that reads Π is If, where $\Pi, p \vdash e$ bool gives $\Pi \vdash e[b/p]$ bool. *At an* `if`, a closed configuration has $\Pi = \cdot$, so e is closed and $[\![e]\!]$ a bit, and both arms are typed in the same Ω at the same A, so the arm it selects is. □

Proof of theorem 5.2. (1) ⇔ (2) is lemma 4.3(1), a per-record map being a contraction. (3) ⇒ (1): on the kept record the block carries the fixed vector $R\psi$ and the site map is $R\psi \otimes W'$

with $W'x = (\mathrm{id} \otimes \langle 0|_a)U(x \otimes |0\cdots 0\rangle \otimes |0\rangle_a)$, the middle factor supplying the other temporaries. Thus s is safe on o with certificate $[R\psi]$. (2) ⇒ (3): with $\sigma_s^o = g \otimes W_o$ and W_o a contraction, put $D = (I - W_o^\dagger W_o)^{1/2}$ and let $J : [\![\Omega]\!] \to \mathcal{H}_{\mathrm{rest}}$ append the other temporaries in $|0\rangle$. Then $x \mapsto W_o x \otimes |0\rangle + JDx \otimes |1\rangle$ is an isometry into $\mathcal{H}_{\mathrm{rest}} \otimes \mathbb{C}^2$, which lemma 4.6 extends to a unitary U of that register, the inputs embedded in $|0\cdots 0\rangle$ as in theorem 4.10. The record on which a reads 0 leaves $g \otimes W_o x$, and lemma 4.6 supplies a gate R with $R\psi = g$. (3) ⇒ (4): equal maps, equal answers. (4) ⇒ (2): lemma 4.9, along o, gives $\sigma_s^o = c\,(R\psi \otimes W') = R\psi \otimes cW'$ with $|c| = 1$ when the maps are nonzero. If they are zero take $c = 0$. In either case this has the shape of (2). □

Proof of corollary 5.3. Both prefixes denote linear maps along o, so lemma 4.9 gives $\sigma'^o = c\,\sigma^o$, with $|c| = 1$ once $\sigma^o \neq 0$. Safety is blind to weights: if $\sigma^o = g \otimes W$ then $c\,\sigma^o = g \otimes cW$, so each certificate of one site map is a certificate of the other and the two sites are safe or unsafe together. At an unreachable record both maps are zero, and the zero site map is safe with every unit vector of the block as certificate. □

Proof of theorem 5.6. Typing follows the displayed statement sequence; the unused parameter p is allowed. With the temporary first, the state before the lift is $((x_0 + x_1)|00\rangle + (x_0 - x_1)|11\rangle)/\sqrt{2}$. The two projections keep respectively its first and second terms, each with weight $1/2$ at either basis input.

The gate V fixes $|00\rangle$ and maps $|11\rangle$ to $(|01\rangle + |10\rangle)/\sqrt{2}$, giving the two site maps displayed in the theorem. The first factors with certificate $|0\rangle$. For the second, the Kraus operators are $|1\rangle\langle u|$ and $|0\rangle\langle u|$, with $\langle u| = (1, -1)/2$. They are nonzero and not proportional, so the site is unsafe. Finally, V is unitary: its four images are $|00\rangle$, $|11\rangle$ and $(|01\rangle \mp |10\rangle)/\sqrt{2}$, an orthonormal basis. □

Here is the program section 5.6 promised, containing the witness of theorem 5.6 verbatim and safe on its only reachable record. Allocate a fresh register r in $|0\rangle$, bind p by lifting it against $\{0\}$, and drop r. The outcome is always 1. Put program (c)'s body in the branch guarded by $p \wedge \neg p$. In the other branch, allocate t in $|0\rangle$, drop it, and return the original input. Both branches type, but only the second is reachable; its drops are safe.

Proof of proposition 5.8. Both programs allocate a temporary qubit on $|0\rangle$ beside a visible qubit, apply H to the temporary — reaching $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes x$ — and drop the temporary at the end.

The first then applies CZ and lifts against $\{0\}$. CZ gives $\frac{1}{\sqrt{2}}(|0\rangle \otimes x + |1\rangle \otimes Zx)$, so the record taking 1 has site map $\frac{1}{\sqrt{2}}|0\rangle \otimes I$ and the record taking 0 has $\frac{1}{\sqrt{2}}|1\rangle \otimes Z$: each is of the form $g \otimes W$, so both drops are safe, with subchannels $\rho \mapsto \frac{1}{2}\rho$ and $\rho \mapsto \frac{1}{2}Z\rho Z^\dagger$.

The second lifts against $\{0\}$ first and only then applies the controlled write $t := t \oplus v$, the visible qubit controlling. The record taking b keeps $\frac{1}{\sqrt{2}}|1 \oplus b\rangle \otimes x$, and the write sends it to $\frac{1}{\sqrt{2}}\sum_c x_c|1 \oplus b \oplus c\rangle \otimes |c\rangle$, whose Kraus operators are the two rank-one projections P_0, P_1 of the visible qubit, each scaled by $\frac{1}{\sqrt{2}}$. These are not proportional, so both drops are unsafe, and both subchannels are $\rho \mapsto \frac{1}{2}\,\mathrm{diag}(\rho)$.

Both pairs sum to $\rho \mapsto \mathrm{diag}(\rho)$: $\frac{1}{2}(\rho + Z\rho Z^\dagger)$ kills the off-diagonal entries and keeps the diagonal, and $\frac{1}{2}\,\mathrm{diag}(\rho)$ twice is the same map. Every branch weight is $\mathrm{tr}(\rho)/2$ at every input on both sides, each of the four site maps above being $\frac{1}{\sqrt{2}}$ times an isometry. □

Proof of theorem 5.10. Fix o and induct on the surface program as theorem 3.5 does, read on the branch the classical register records as o; that proof's three measurement-free clauses and its Drop clause carry over, so the work is the three new clauses and the reclaiming gates.

A single-level lift on a qubit register. The emitter writes `c[k] = measure q` and reads the record's bit b as `c[k] == j` for the level $\{j\}$, so that $b = 1$ means the measured value is j, whichever level is named. On the branch where the bit is b the map on the surviving amplitudes is the projection P^b on q, which is what Step-Lift applies; the bit is left in the classical register, where the record lives.

A lift against a larger block. By remark 5.4 the source has the same per-record map as the derived program up to the factor $|b\rangle$ on the fresh register a, whose own drop is safe with certificate $|b\rangle$. The emitter writes the indicator permutation U_S, then that same measurement, then X^b on a conditioned on the bit, which is proposition 4.11's gate for that certificate. So the emitted branch map is $x \mapsto E_o(M)x \otimes |0\rangle_a$. Omitting this additional fixed factor gives the map in the statement.

An `if` is OpenQASM's classical conditional, which selects a branch and applies nothing to the quantum state: Step-If.

The reclaiming gates. Let $s_1, \dots, s_r$ fire and be safe on o, in that order. At s_j the emitter writes $R_o^{s_j}$ conditioned on the bits of o taken before s_j. These bits suffice to select the certificate by proposition 5.9(1), whether or not the source program tests them (2). The emitted condition may mention earlier bits that are unnecessary for selecting this particular gate.

From s_j onwards s_j's registers are factors of $\mathcal{H}^o_{\mathsf{hid}}$ that no later clause names, so $R_o^{s_j}$ commutes past everything emitted after it and collects at the end: the displayed branch map. Clauses (1) and (2) follow from this display. Clause (3) uses the temporary and loan restoration arguments in the proof of theorem 3.5. □

Proof of proposition 5.9. (1) The machine is deterministic once its labels are fixed, so the prefix of the run up to s — and with it σ_s^o — is a function of the bits taken before s. By hypothesis $\sigma_s^o \neq 0$, so lemma 4.3(1) determines g_o from σ_s^o up to a phase, and proposition 4.11's tie-break fixes the representative, hence R_o. (2) Allocate a and x, put them on a Bell pair, lift x against $\{0\}$, and drop a without testing the bit: `new a : [2] @ |0>; new x : [2] @ |0>; H a; CX a x; p = lift[{0}] x; drop a; return x`. The Bell pair is $(|00\rangle + |11\rangle)/\sqrt{2}$ on (a, x), so the record taking 1 leaves a on $|0\rangle$ and the record taking 0 leaves it on $|1\rangle$: the site is safe on both, with reclaiming gates I and X, and the text between the lift and the drop is the same on both, and the variant that tests the bit before the drop has the same two certificates and the same two gates. □

D Proofs for the ring and cost sections

Throughout, $\omega = e^{i\pi/4}$, $\mathcal{R} = \mathbb{Z}[1/\sqrt{2}, i] = \mathbb{Z}[\omega][1/\sqrt{2}]$, and $\mathcal{R}_0 = \mathbb{Z}[1/\sqrt{2}]$ is its real subring.

Fact D.1 (the column lemma [1, 13]). *A ring $\mathcal{A}$ of complex numbers closed under conjugation has a* column lemma *when, for every $n \geq 1$ and every unit vector $v \in \mathcal{A}^n$, some unitary over $\mathcal{A}$ carries v to e_1. The following five rings have this property. The middle column gives a gate set with that coefficient ring; the last locates the column lemma. Here* CH *is controlled Hadamard and $F = (H + iI)/\sqrt{2}$ [1, §4].*

Ring	*Gate set*	*Source*
$\mathbb{Z}[1/\sqrt{2}, i]$	*Clifford+T*	*[13, Lemma 5]*
$\mathbb{Z}[1/2]$	$X, \mathrm{CX}, \mathrm{CCX}, H \otimes H$	*[1, Lemma 5.4]*
$\mathbb{Z}[1/\sqrt{2}]$	$X, \mathrm{CX}, \mathrm{CCX}, H, \mathrm{CH}$	*[1, Lemma 5.14]*
$\mathbb{Z}[i/\sqrt{2}]$	$X, \mathrm{CX}, \mathrm{CCX}, F$	*[1, Lemma 5.19]*
$\mathbb{Z}[1/2, i]$	$X, \mathrm{CX}, \mathrm{CCX}, \omega H, S$	*[1, Lemma 5.25]*

The proof uses three facts about $\mathcal{R}$ and its real subring $\mathcal{R}_0$. First, $\mathcal{R}$ is a principal ideal domain: $\mathbb{Z}[\omega]$ is the ring of integers of $\mathbb{Q}(\zeta_8)$, whose class number is one [42, Ch. 11], and a localisation of a principal ideal domain is one. Second, the units of $\mathcal{R}_0$ are $\pm\sqrt{2}^a(1+\sqrt{2})^b$ with $a, b \in \mathbb{Z}$: the

units of $\mathbb{Z}[\sqrt{2}]$ are $\pm(1+\sqrt{2})^b$, and inverting the prime $\sqrt{2}$ adds one generator. Third, three norms: $|\sqrt{2}|^2 = 2$, $|1+\sqrt{2}|^2 = (1+\sqrt{2})^2$, and $|1+\omega|^2 = 2+\omega+\bar{\omega} = 2+\sqrt{2} = \sqrt{2}(1+\sqrt{2})$, where $1+\omega$ is a unit of $\mathcal{R}$ because $(2+\sqrt{2})(2-\sqrt{2}) = 2$ and $2 = (\sqrt{2})^2$ is a unit. Write ς for the automorphism of $\mathbb{Q}(\omega)$ defined by $\varsigma(\omega) = \omega^5$, so $\varsigma(\sqrt{2}) = -\sqrt{2}$; the Galois group is abelian, so ς commutes with complex conjugation and $\varsigma(|x|^2) = |\varsigma(x)|^2 \geq 0$.

Proof of theorem 6.2. Part (2): if g is over $\mathcal{R}$ so is $gg^\dagger$, the ring being closed under conjugation; if $gg^\dagger$ has entries in $\mathcal{R}$, part (1) writes it $zz^\dagger$ with z over $\mathcal{R}$, and $zz^\dagger = gg^\dagger$ forces $z = \lambda g$ with $|\lambda| = 1$. For the claim about a safe site, suppose the surviving weight at a basis input e_k is a unit of $\mathcal{R}$. The vector $\sigma_s^o(e_k) = g \otimes We_k$ has entries in $\mathcal{R}$. So with $w = We_k$ every $g_i\bar{g}_j \|w\|^2 = \sum_q (g_i w_q)\overline{(g_j w_q)}$ lies in $\mathcal{R}$, and $\|w\|^2$ is the record's surviving weight at e_k. That weight is a unit, so $g_i\bar{g}_j$ lies in $\mathcal{R}$ too, and part (2) gives a unit vector over $\mathcal{R}$ on the certificate. On the one record of a measurement-free program σ_s is an isometry, the weight is 1, and theorem 6.1(2), which applies because $\mathcal{R}$ has a column lemma, gives the witness; a witness is safe by theorem 4.10. So the work is in (1). Let $P = gg^\dagger$ with $\|g\| = 1$ have entries in $\mathcal{R}$. Some column $r = Pe_j = \bar{g}_j\, g$ is nonzero; it lies in $\mathcal{R}^d$ and $P = rr^\dagger/\|r\|^2$. Divide r by a greatest common divisor of its entries, which exists because $\mathcal{R}$ is a principal ideal domain: $r = \delta r'$ with r' primitive, and still $P = r'r'^\dagger/N$ for $N = \|r'\|^2 = \sum_i |r'_i|^2 \in \mathcal{R}_0$. Since $NP_{ik} = r'_i\bar{r}'_k$, N divides every product $r'_i\bar{r}'_k$, and those products generate $(r'_0, \ldots, r'_{d-1})(\bar{r}'_0, \ldots, \bar{r}'_{d-1}) = (1)$; so N is a unit of $\mathcal{R}$ and, being real, of $\mathcal{R}_0$, say $N = \sqrt{2}^a(1+\sqrt{2})^b$, positive. Then $\varsigma(N) = \sum_i |\varsigma(r'_i)|^2$ is positive too, while $\varsigma(N) = (-\sqrt{2})^a(1-\sqrt{2})^b$ has the sign of $(-1)^{a+b}$, so $a \equiv b \pmod 2$. Put $c = a \bmod 2$ and

$$\varepsilon \;=\; \sqrt{2}^{\,(a-c)/2}\,(1+\sqrt{2})^{(b-c)/2}\,(1+\omega)^c,$$

a unit of $\mathcal{R}$ with $|\varepsilon|^2 = \sqrt{2}^{\,a-c}(1+\sqrt{2})^{b-c}\big(\sqrt{2}(1+\sqrt{2})\big)^c = N$. Then $z = r'/\varepsilon$ has entries in $\mathcal{R}$, $\|z\|^2 = N/|\varepsilon|^2 = 1$ and $zz^\dagger = r'r'^\dagger/N = P$. At $d = 1$ the steps degenerate rather than fail: $P = (1)$, $r' = 1$, $N = 1$, $a = b = c = 0$, $\varepsilon = 1$ and $z = 1$. □

Safety alone does not put a certificate over $\mathcal{R}$. The exact condition is that its projection have entries in $\mathcal{R}$, as in theorem 6.2(2). Allocate a two-qubit temporary on $|00\rangle$, apply $H \otimes H$, and lift it against three of its four basis states. On the record that survives, of weight $\frac{3}{4}$, which is not a unit of $\mathcal{R}$, the drop is safe with certificate $(1,1,1,0)/\sqrt{3}$, whose projection has the entry $\frac{1}{3} \notin \mathcal{R}$: no unit vector over $\mathcal{R}$ lies on that ray, and no gate over $\mathcal{R}$ prepares it. A unit surviving weight is sufficient but not necessary: a block left on $|+\rangle$ beside two qubits lifted the same way has weight $\frac{3}{4}$ too, and its certificate is over $\mathcal{R}$, its projection being so.

Proof of proposition 6.3. The proof above used three properties of $\mathcal{R}$: a principal ideal domain closed under conjugation, a real subring, and every totally positive unit of that subring a norm $|\varepsilon|^2$ of a unit. For $\mathbb{Z}[1/2, i] = \mathbb{Z}[i][1/2]$: $\mathbb{Z}[i]$ has class number one, the real elements form $\mathbb{Z}[1/2]$, whose positive units are the 2^a, and $2^a = |(1+i)^a|^2$ with $1+i$ a unit since $2 = -i(1+i)^2$. For $\mathbb{Z}[i/\sqrt{2}] = \mathbb{Z}[\sqrt{-2}][1/2]$: $\mathbb{Z}[\sqrt{-2}]$ has class number one, the real elements again form $\mathbb{Z}[1/2]$, and $2^a = |\sqrt{-2}^{\,a}|^2$ with $\sqrt{-2}$ a unit since $2 = -(\sqrt{-2})^2$. Both class numbers are one because the Minkowski bound of each of $\mathbb{Q}(i)$ and $\mathbb{Q}(\sqrt{-2})$ is below 2.

The two that do not split. Over a real ring $\mathcal{A}$ a splitting vector z of a real projection $P = gg^\top$ is λg with $|\lambda| = 1$, and z, g real force $\lambda = \pm 1$, so $z = \pm g$, and the question becomes whether the ray itself carries a unit vector over the ring. Over $\mathbb{Z}[1/2]$ the projection $P = \frac{1}{2}\left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ has entries in the ring and $\pm(1,1)/\sqrt{2}$ does not. Over $\mathbb{Z}[1/\sqrt{2}]$ take the H-type magic state $g = \cos\frac{\pi}{8}|0\rangle + \sin\frac{\pi}{8}|1\rangle$, whose ray is $[(1+\sqrt{2},\ 1)]$ and whose projection $\frac{1}{2}(I + (X+Z)/\sqrt{2})$ has entries in the ring. A unit vector

on that ray is $\lambda(1+\sqrt{2},1)$ with $\lambda^2 = (2-\sqrt{2})/4$, whose norm down to $\mathbb{Q}$ is $1/8$, not a rational square, so no such λ lies in $\mathbb{Q}(\sqrt{2})$, let alone in $\mathbb{Z}[1/\sqrt{2}]$. Over $\mathcal{R}$ it splits, with $z = \left(\frac{1+\omega}{2}, \frac{i(1-\omega)}{2}\right)$.

The one-gate program of section 6.2: the entries of $rot \otimes rot$ are $\frac{2+\sqrt{2}}{4}$, $\pm\frac{2-\sqrt{2}}{4}$ and $\pm\frac{\sqrt{2}}{4}$, all in $\mathbb{Z}[1/\sqrt{2}]$, and $(rot \otimes rot)(|0\rangle \otimes x) = g \otimes rot\, x$ with $g = rot|0\rangle$ the magic state above. □

Proof of theorem 7.1. *Membership.* Every gate of the set permutes the standard basis, a clean allocation adjoins a basis vector, and a lift keeps a basis vector or sends it to zero. So along the record o the site map sends each basis input e_x either to zero — x does not survive the record — or to a basis vector $|f(x)\rangle \otimes |h(x)\rangle$, block first. Both f and h are computed by running the program on x in time linear in its size, block membership at a lift being read off the block's presentation. If f is constant, $f \equiv f_0$, on the surviving inputs, then $\sigma_s^o = |f_0\rangle \otimes W$ with $We_x = |h(x)\rangle$ or 0, and the site is safe by definition 4.1; when no input survives, $\sigma_s^o = 0$ and any f_0 serves. If two surviving inputs x, x' have $f(x) \neq f(x')$, then $\sigma_s^o e_x$ and $\sigma_s^o e_{x'}$ have different block factors, which no $g \otimes W$ allows, and the site is unsafe. So the site is unsafe on o exactly when such a pair (x, x') exists, a witness checked in polynomial time: the unsafe instances are in NP and the safe ones in coNP.

Hardness. Reduce from unsatisfiability. Let B be a Boolean circuit over $\{\wedge, \neg\}$ with $n \geq 1$ inputs and s gates — a CNF formula converts to one in polynomial size, and $n \geq 1$ costs nothing, a formula with no variables being decided by one evaluation while step three needs a nonempty input context.

Step one: evaluate B reversibly, by Bennett's compute–copy–uncompute [4]. Allocate s scratch qubits and one target at $|0\rangle$ and, in topological order, write each gate of B into a *fresh* scratch qubit. For $u \wedge w$ with distinct operands, use a Toffoli controlled by u, w; for $u \wedge u = u$, use a CNT controlled by u. For $\neg u$, use a CNT from u into the fresh scratch qubit, followed by an X on that scratch qubit. A CNT then copies the output qubit into the target, and every gate before the copy is applied again in reverse order; each is an involution, so the scratch returns to $|0\rangle$ at every input and the net map is $|x\rangle \otimes |0^s, 0\rangle \mapsto |x\rangle \otimes |0^s, B(x)\rangle$.

Step two: read the verdict off it. Take the block to be the $s+1$ scratch and target qubits, allocated at $|0^{s+1}\rangle$, and the input context to be x: a program of size $O(n+s)$ with $s+1$ clean allocations and one `drop`, whose site map is $\sigma_s(e_x) = |0^s, B(x)\rangle \otimes e_x$. By definition 4.1 it is safe exactly when the vectors $|0^s, B(x)\rangle$ all lie on one ray, that is exactly when B is constant.

Step three: constancy into unsatisfiability. Compute $B(0^n)$. If it is 1, output a fixed unsafe program — a CNT from the first input qubit into a fresh one-qubit block — and B is satisfiable. If it is 0, output step two's program, safe exactly when B is constant and then constantly 0, hence exactly when B is unsatisfiable. Either way the program produced is safe if and only if B is unsatisfiable, and the reduction runs in polynomial time. With the membership half, the problem is coNP-complete. □

Proof of theorem 6.1. (1) A witness with R over $\mathcal{A}$ denotes $R\psi \otimes W(- \otimes |0 \cdots 0\rangle)$, so the site is safe by theorem 4.10 and its certificate is the ray of $R\psi$, a unit vector over $\mathcal{A}$ because R and ψ are.

(2) *Claim: if $\mathcal{A}$ has a column lemma and $v_1, \ldots, v_k \in \mathcal{A}^n$ are orthonormal, some unitary over $\mathcal{A}$ has them as its first k columns.* By induction on k. At $k = 0$ take $U = I$. For the step, fact D.1 gives a unitary G over $\mathcal{A}$ with $Gv_1 = e_1$. For $j \geq 2$, $\langle e_1, Gv_j\rangle = \langle Gv_1, Gv_j\rangle = \langle v_1, v_j\rangle = 0$, so $Gv_j = (0, v_j')$ with $v_j' \in \mathcal{A}^{n-1}$, its entries lying in $\mathcal{A}$ because those of G and v_j do; and $v_2', \ldots, v_k'$ are orthonormal. By the induction hypothesis some unitary U' over $\mathcal{A}$ has them as its first $k-1$ columns. Put $U = G^\dagger(1 \oplus U')$, which is over $\mathcal{A}$ because $\mathcal{A}$ is closed under conjugation. Then $Ue_1 = G^\dagger e_1 = v_1$ and $Ue_j = G^\dagger(0, v_j') = G^\dagger G v_j = v_j$.

Now let s be safe and g a unit vector over $\mathcal{A}$ on its certificate ray, so $\sigma_s = g \otimes W$ with $W = (\langle g| \otimes I) \circ \sigma_s$ by lemma 4.3(1). Every entry of σ_s is a sum of products of entries of M's gates and allocation vectors, hence lies in $\mathcal{A}$, and so does $W_{jk} = \sum_i \bar{g}_i\, (\sigma_s)_{(i,j),k}$, the ring being closed under

conjugation. The columns of W are orthonormal and over $\mathcal{A}$, and a permutation of the standard basis — a unitary over every ring — places them at the basis of $[\![\Omega]\!] \otimes |0 \cdots 0\rangle$, a subspace of $\mathcal{H}_{\text{rest}}$ because the block is temporaries; so the claim extends W to a unitary over $\mathcal{A}$. For the block, the claim gives unitaries A and B over $\mathcal{A}$ whose first columns are ψ and g, and $R = BA^\dagger$ is over $\mathcal{A}$ with $R\psi = g$. Then $\mathrm{sep}(R, W)$ is a program over $\mathcal{A}$ and denotes $g \otimes W = \sigma_s$. □

A reclaiming unitary R and allocation ψ over $\mathcal{A}$ give the unit certificate $R^\dagger\psi$. With a column lemma this also suffices, by the preceding proof. *The reflection* R^ψ itself is over $\mathcal{A}$ if the chosen representative is and $\|w\|^2$ is zero or a unit of $\mathcal{A}$. Over $\mathbb{Z}[1/2, i]$ with $g = \left(\frac{1+i}{2}, \frac{1-i}{2}\right)$ and $\psi = |0\rangle$ the ring splits $gg^\dagger$ and the completion $\left(\begin{smallmatrix} (1+i)/2 & -(1+i)/2 \\ (1-i)/2 & (1-i)/2 \end{smallmatrix}\right)$ is a unitary over it whose first column is g, so its adjoint carries g to e_0, while the representative proposition 4.11 chooses is $(1/\sqrt{2}, -i/\sqrt{2})$ and its reflection has entries $\pm 1/\sqrt{2}$ and $\pm i/\sqrt{2}$, which the ring does not contain.